



Université catholique de Louvain
École Polytechnique de Louvain
Pôle d'ingénierie informatique



Application de la programmation par contraintes au problème de chargement de chimiquier.

Promoteur : Pierre Schaus
Lecteurs : Vianney le Clément
Yves Deville

Mémoire présenté en vue de l'obtention
du grade de
Master ingénieur civil en informatique
option intelligence artificielle
par François Pelsser

Table des matières

1	Environnement	4
1.1	Outils	4
1.1.1	Scala	4
1.1.2	Oscar	5
1.1.3	Visuels	6
2	Routage de chimiquier	8
2.1	Introduction	8
2.2	Problème d'allocation des cuves	9
2.3	Programmation linéaire	11
2.3.1	Modèle	11
2.3.2	Implémentation	13
2.4	Programmation par contraintes	16
2.4.1	Contraintes	17
2.4.2	Recherche	19
2.4.3	Implémentation	19
2.4.4	Allocation des volumes	22
2.5	Expérimentation	22
2.5.1	Génération des instances de test.	22
2.5.2	Benchmarks	25
3	BinPacking constraint	30
3.1	Introduction	30
3.2	Contrainte BinPacking	30
3.3	Réflexion sur les cardinalités	31
3.4	Amélioration du filtrage de la contrainte	32
3.4.1	Filtrage sur les variables de cardinalité	32
3.4.2	Filtrage sur les variables de capacité	35
3.5	Expérimentations	37
3.6	Combiner des boîtes pour des bornes encore plus fortes	38
3.7	Conclusion	39

Remerciements

Avant toute chose, je tiens à remercier Monsieur Pierre Schaus de m'avoir guidé et conseillé tout au long de la réalisation de ce mémoire et lui adresser ma reconnaissance pour son importante collaboration lors de la rédaction de l'article. Je souhaite également marquer ma gratitude envers l'ensemble des professeurs pour leurs précieux enseignements.

Je tiens également à remercier Monsieur Jean-Charles Régin pour sa collaboration à l'article découlant de ce mémoire.

Je remercie également Monsieur Vianney le Clément et Monsieur Yves Deville pour avoir accepté d'être les lecteurs de ce mémoire.

Un remerciement particulier va également à mes relecteurs vigilants qui ont accepté de prendre le temps de corriger ce travail.

Un tout grand merci à ma famille et à mes amis de Baelen et de Louvain-la-Neuve qui m'ont soutenu et encouragé jour après jour et plus spécialement à Amélie qui était présente et toujours disponible au quotidien.

Enfin, je voudrais dédier ce mémoire à mes parents et à Amélie. Ils m'ont épaulé tout au long de ces années d'études. J'espère que ce travail pourra les rendre fiers de moi.

Introduction

Durant notre cursus universitaire, nous avons goûté au plaisir de l'optimisation grâce à la mineure en mathématiques. Plus tard, nous avons eu un cours d'intelligence artificielle qui nous a permis de découvrir une branche de l'informatique qui nous plaisait. Ce cours et notre affection pour les mathématiques ont guidé notre orientation vers l'option Intelligence Artificielle. La prochaine étape qui nous a amené à choisir ce mémoire est le cours de programmation par contraintes. Ce dernier cours était composé d'une série de travaux qui nous ont fait apprécier cette partie de la recherche opérationnelle.

Ceci induit que, lors de la recherche du thème de notre mémoire, nous nous sommes tourné vers un sujet contenant de la recherche opérationnelle et, si possible, de la programmation par contrainte. De tous les sujets possédant ces aspects, celui du problème de routage de chimiquier nous a plu pour son côté réaliste.

Les informations à propos du problème de routage de chimiquier tel qu'il existe à l'heure actuelle sont principalement issues de [?]. En plus des problèmes sur lesquels nous pourrions travailler, d'autres informations viennent de [3]. Ce dernier article s'accompagne d'une modélisation mathématique d'une partie du problème de routage qui est l'allocation de produits aux cuves d'un bateau le long d'un trajet. Grâce à cet article nous possédions un modèle et des instances du problème sur lesquels nous pouvions travailler. Pour l'article, la méthode utilisée pour résoudre ces instances était la programmation linéaire. Nous avons, pour commencer, répliqué cette méthode et nous avons ensuite réalisé un modèle utilisant la programmation par contraintes pour résoudre ces mêmes instances.

Pour répliquer ce modèle et créer celui utilisant la programmation par contraintes, nous avons utilisé Oscar [5]. Cet outil nous offrait la possibilité d'implémenter ces deux modèles différents et de les résoudre. Le langage utilisé pour créer ces modèles est Scala¹.

Pour modéliser le modèle utilisant la programmation par contrainte, nous nous sommes inspiré de [7] et, plus précisément, de l'adaptation de la contrainte *BinPacking*. Nous nous sommes alors un peu distancé du problème original pour faire évoluer cette contrainte en une version possédant un meilleur filtrage. Cette nouvelle contrainte n'étant pas efficace pour les problèmes d'allocation de cuves, nous nous sommes tourné vers le problème *Balanced Academic Curriculum Problem*. Un article a été écrit sur cette évolution de la contrainte avec la collaboration du Professeur Pierre Schaus et du Professeur Jean-Charles Régin.

Les sources réalisées pour ce mémoire sont disponible à l'adresse <http://bitbucket.org/fpelsser/memoire>. Une explication plus détaillé de l'organisation de ces sources est disponible en annexe.

1. <http://www.scala-lang.org/>

Chapitre 1

Environnement

1.1 Outils

1.1.1 Scala

Scala¹ est un langage de programmation multi-usages, conçu pour pouvoir implémenter de nombreux modèles de programmations de manière élégante, concise et permettant de garder la sécurité du typage. Son nom provient de scalabilité, ce qui signifie qu'il permet d'écrire toutes sortes de programmes : du script à l'application entreprise. Cette capacité est très importante lors de la phase de modélisation, de plus, de nombreux tests relevant de différentes techniques doivent être effectués. Il permet en effet d'exprimer ces tests très rapidement et avec très peu de lignes de code. Il fait également évoluer ce code d'un simple test à un élément plus générique qui sera utilisé très facilement et de manière intensive.

Comme les applications Scala utilisent la JVM (machine virtuelle Java), elles fonctionneront sur tout ordinateur capable de lancer des applications Java. De plus, Scala est interopérable avec toutes les bibliothèques et tout élément déjà écrit en Java. Cette interopérabilité fait de Scala, bien qu'il soit encore assez jeune, un langage très riche avec des API très complètes et robustes. Les applications Scala peuvent dès lors contenir des parties écrites en Java et d'autres en Scala. Par exemple, la bibliothèque Oscar (voir section 1.1.2) succède à la bibliothèque Scampi qui était écrite en Java et contient à l'heure actuelle des parties de sources en Java et d'autres en Scala.

Scala est un langage qui combine le paradigme orienté-objet et fonctionnel. Entre autres, ceci réduit considérablement le nombre de lignes nécessaires pour exprimer une fonctionnalité.

Selon la documentation Scala, Java nécessite 2 à 3 fois plus de lignes de code que Scala pour un même but.

Un autre facteur qui permet d'exprimer une idée de façon très concise et élégante en Scala est le grand nombre d'abstractions disponibles. Par exemple, Scala possède un système d'inférence de type qui permet, la plupart du temps, de s'abstenir de définir explicitement le type des variables tout en restant strictement typé et donc, en gardant toutes les sécurités que cela apporte. Scala peut également inférer le type du retour d'une méthode. Dans l'exemple 1.1.1, le système de types va trouver, lors de la compilation, que `pi` est de type `Double` et que la méthode `square` retourne un `Int`.

Le paradigme fonctionnel permet d'écrire des éléments beaucoup plus génériques. Les opérations sur des listes sont de bons exemples de ce comportement fonctionnel. La méthode

1. <http://www.scala-lang.org/>

Exemple 1.1.1 Inférence de type en scala

```
1 val pi = 3.1415
2 val square(i: Int) = i * i
```

map permet, à partir d'une fonction donnée par l'utilisateur de générer, pour tout élément de la liste, un nouvel élément. En appliquant cette méthode sur une liste, on obtient, dès lors, une nouvelle liste où tout élément est le résultat de l'application de la fonction sur l'élément correspondant dans la liste initiale. Il existe de nombreuses opérations de ce genre telles que le tri de listes, le calcul de sommes d'éléments ou du nombre d'éléments validant une expression,... L'exemple 1.1.2 montre un script qui retourne une liste des carrés des 100 premiers entiers et met en place un filtre afin de n'avoir que les éléments pairs.

Exemple 1.1.2 Opération sur les listes

```
1 (1 to 100).map(i=>i * i).filter(_%2==1)
```

Grâce à la richesse de Scala et à ce comportement fonctionnel, la parallélisation peut être faite lors de l'opération sur des listes de manière très facile. Dans l'exemple 1.1.3, une liste contient des objets qui doivent être hashés. La fonction de hashage est conséquente. Ainsi, il serait intéressant de traiter la liste sur plusieurs threads. La méthode *par*, appliquée à la liste *objects*, la prépare pour la parallélisation. Dès lors, lorsque la méthode *map* est exécutée, Scala s'occupe d'appliquer la méthode fournie à *map* sur les éléments de manière parallèle et de renvoyer une liste avec les résultats de cette exécution.

Exemple 1.1.3

```
1 val hash = objects.par.map(_.hash)
```

Un dernier aspect intéressant de Scala est sa syntaxe riche et très modulable. Cela permet d'écrire du code source de manière très expressive. Par exemple, les méthodes avec 0 ou 1 paramètre peuvent être appelées sans le point entre l'objet et le nom de la méthode et sans mettre le paramètre entre parenthèses. Ceci est représenté dans l'exemple 1.1.4. Etant donné la propriété précédente et le fait que Scala autorise les caractères spéciaux, il est possible, comme en c++ par exemple, de surcharger les opérateurs. Cette richesse d'expression et la réduction du nombre de lignes de code par rapport à des langages traditionnels facilitent l'écriture et la lecture des sources écrites en Scala.

1.1.2 Oscar

Oscar [5] est une "caisse à outils" pour la résolution de problèmes de recherche opérationnelle. Il contient des outils pour la programmation linéaire (et intégrale), la programmation par contraintes, la recherche locale avec contraintes, la simulation à événements discrets et l'optimisation sans dérivées. Il contient également des outils pour visualiser les résultats de manière assez simple.

Oscar est *open source*. Cependant, pour la programmation linéaire, il ne possède pas son propre solveur mais offre une interface avec des solveurs existants. Oscar permet d'utiliser trois solveurs : *lp_solve*², *Gurobi*³ et *Cplex*⁴. De ces trois solveurs, seul *lp_solve* est *open*

2. <http://lpsolve.sourceforge.net/5.5/>

3. <http://www.gurobi.com/>

4. <http://www-01.ibm.com/software/commerce/optimization/cplex-optimizer/>

Exemple 1.1.4 Equivalence d'expressions

```
1 List(1,2,3).toArray //équivalent List(1,2,3).toArray  
2 1 min 3 //équivalent 1.min(3)
```

source. Les deux autres sont des produits commerciaux et ne sont, dès lors, pas fournis de base avec Oscar.

Grâce à la richesse de la syntaxe de Scala, les solveurs peuvent être intégrés de manière parfaitement homogène et peuvent utiliser toutes les fonctionnalités qu'offre Scala comme, par exemple, le vérificateur de types. Cela permet de rendre Oscar très facile à utiliser et les modèles faits avec cet outil, très faciles à lire.

La disponibilité des sources est un atout lorsqu'on réalise un mémoire. En effet, cela nous permet de voir la manière dont tout est implémenté et de faire des modifications si nécessaire.

Comme expliqué dans les chapitres suivants, certains outils et certaines contraintes ont été créés pour faire ce mémoire. Certaines de ces créations étant assez génériques que pour être utilisées dans d'autres contextes font, dès à présent, partie de la librairie.

Comme cette librairie est utilisée et développée par le monde de l'entreprise et le monde universitaire, elle se doit de répondre aux besoins que nécessitent ces deux milieux. Oscar est, dès lors, assez robuste pour pouvoir être utilisé en production. Toutefois, il contient également des techniques à la pointe qui peuvent parfois être expérimentales mais peuvent aussi mener à des outils fonctionnels.

Les contributeurs du projet Oscar sont :

- UCLouvain⁵ et le laboratoire de recherche BeCool⁶
- CETIC⁷ qui développe et maintient la partie de recherche locale
- n-Side⁸ qui utilise Oscar et alloue des ressources pour son développement

1.1.3 Visuels

Les visuels sont utiles pour représenter des informations à propos de l'évolution d'une recherche et/ou des résultats de cette recherche. Comme expliqué dans les sections suivantes, certaines personnes ne font pas confiance aux solutions car elles n'ont pas ou peu de connaissances au niveau des techniques utilisées pour les générer et que les résultats peuvent paraître abstraits. Les visuels peuvent permettre de rendre ces résultats plus concrets.

Non seulement ces visuels peuvent être intéressants pour expliquer des solutions à une autre personne mais peuvent également aider à avoir une meilleure appréciation de résultats ou de l'évolution d'une recherche, ce qui peut aider lors de la réalisation d'un modèle.

Certains visuels ont été développés pour ce mémoire tels que l'intégration des barres d'outils ou des listes des sélections.

La politique derrière les visuels d'Oscar n'est pas de créer une application entière mais de créer très rapidement, à l'aide de quelques lignes de code, des visuels représentant le plus clairement possible des informations. Cependant, comme ils utilisent les paquets visuels standards, les visualisations ne sont pas vraiment limitées et tout élément visuel créé avec la librairie visuelle d'Oscar peut très facilement être inséré dans toute autre application.

5. <http://www.uclouvain.be/>

6. <http://becool.info.ucl.ac.be/>

7. <http://www.cetic.be/>

8. <http://www.n-side.com/>

L'implémentation de la barre d'outils, par exemple, est une surcouche de *JToolBar* et contient une fonction qui permet d'ajouter un bouton avec le label du bouton et l'action à effectuer lors d'un clic sur ledit bouton.

Une visualisation faite avec Oscar est composée d'une fenêtre de type *VisualFrame*. Dans cette fenêtre, on a ensuite des sous-fenêtres qui sont créées en appelant la méthode *createFrame("FrameTitle")* de la fenêtre principale. Après cela, nous pouvons ajouter des éléments comme une visualisation de carte, de graphique ou une aire de dessin. Si nous ajoutons une aire de dessin, nous pouvons, par la suite, ajouter des objets tels que des lignes, des rectangles... Lorsque ces objets seront modifiés (changement de position, de taille, de couleur,...), l'aire de dessin sera automatiquement redessinée.

Pour illustrer ce qui est possible avec les visuels Oscar, nous allons créer une fenêtre avec une sous-fenêtre contenant une aire de dessin. La fenêtre principale contiendra une barre d'outils avec un bouton "foo" qui dessinera un rectangle dans l'aire de dessin de la sous-fenêtre. Le rectangle affichera "foobar" lorsqu'il sera survolé. Cette fenêtre est visible à la figure 1.1

Les sources de cette visualisation sont les suivantes :

```
1 val f = new VisualFrame("Basic");
2 val tb = f.createToolBar();
3 val inf = f.createFrame("Rectangle");
4 val d = new VisualDrawing(false);
5 inf.add(d);
6
7 tb.addButton("foo",{
8     val rect = new VisualRectangle(d, 50, 50, 100, 50);
9     rect.setToolTip = "foobar";
10 })
```

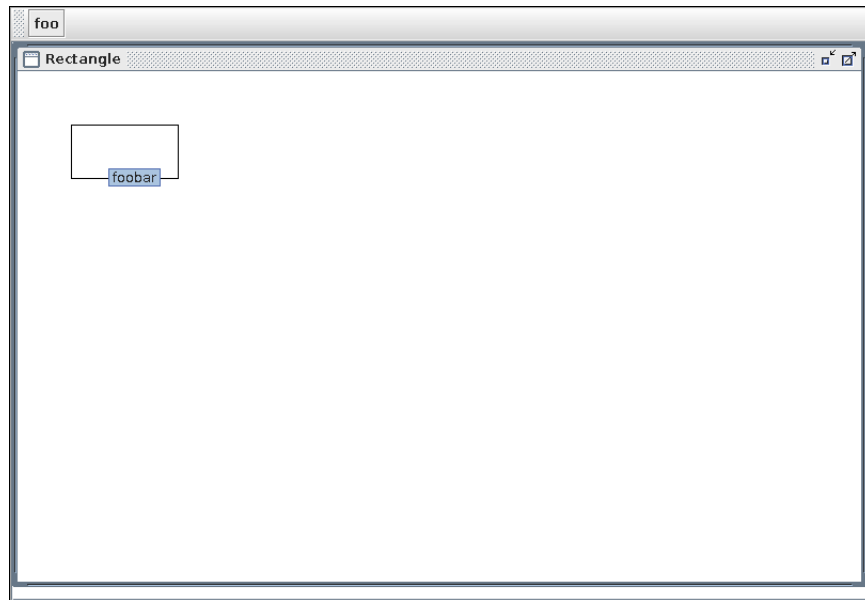


FIGURE 1.1: Exemple de visualisation

Chapitre 2

Routage de chimiquier

2.1 Introduction

Le routage de chimiquier est un problème qui mérite de l'attention. En effet, vu le nombre de ces bateaux et le coût des opérations leur incombant, un planning amélioré serait une importante source d'économies. Ce problème est, par contre, très compliqué à résoudre par le nombre important de contraintes liées au placement de produits dans le bateau et à la navigation de ce dernier. De plus, certaines contraintes sont compliquées à modéliser. Cela rend impossible la résolution du problème de manière complètement automatique.

Kjetil Fagerholt, dans [?], explique les différents enjeux et problématiques de ce problème. Il explique également que dans le logiciel *TurboRouter*¹, ils se sont principalement concentrés sur l'interface permettant de préparer les routages plutôt que sur les algorithmes permettant de les automatiser et ce, en raison de la réticence des utilisateurs de laisser cette tâche à un outil automatisé.

A l'heure actuelle, le routage est réalisé majoritairement par des humains avec, parfois, l'aide de systèmes d'aide à la décision. Le planificateur se voit attribuer une flotte (ou un segment de flotte) et des cargos à livrer. Certains cargos sont obligatoires, d'autres doivent être négociés par l'opérateur.

Son rôle est de maximiser le bénéfice apporté par cette flotte. Cela implique de maximiser le nombre de cargos transportés tout en minimisant les coûts. Ces coûts peuvent être nombreux et parfois ne pas être calculables. Par exemple, charger le bateau de telle sorte que les cuves les plus polyvalentes soient libres le plus fréquemment possible, ou charger de telle sorte que la stabilité de ce bateau, à tout moment, possède les plus grandes marges possibles, permettra éventuellement de répondre à un cargo négociable qui ne peut être prévu pendant l'optimisation.

Une flotte est un ensemble de navires décrits par plusieurs attributs. Premièrement, il faut renseigner la capacité de chaque cuve, aussi bien en volume qu'en poids. Chaque cuve, quant à elle, s'accompagne d'une série de contraintes telles que le volume minimal qui peut lui être assigné afin d'éviter le ballotement, les types de produits qui peuvent lui être assignés... Ensuite, il est nécessaire de prendre en compte le modèle de consommation de carburant du bateau, la vitesse de navigation ainsi que les vitesses de chargement et de déchargement des biens. Finalement, chaque cuve ainsi que son contenu ont une influence sur la stabilité du bateau. Les limites de stabilité du bateau doivent donc également être prises en compte.

1. <http://www.sintef.no/Projectweb/TurboRouter/>

Comme indiqué, les cargos peuvent parvenir de deux manières différentes. Certains cargos sont issus de contrats à longs termes avec l'expéditeur et doivent obligatoirement être livrés. Les autres cargos sont issus de contrats au comptant proposés par des courtiers. Une fois la proposition reçue, l'opérateur décide ou non de négocier le cargo. Si le cargo est accepté, il doit être traité comme un cargo issu d'un contrat et doit donc obligatoirement être délivré.

Les cargos s'accompagnent également d'une liste d'attributs. Premièrement, chaque cargo arrive en une certaine quantité et possède une masse volumique. Les cargos ont également un type et ils requièrent certaines propriétés pour les cuves qui vont les contenir. Ces propriétés peuvent être la nature du revêtement de la cuve ou la capacité d'en chauffer ou d'en refroidir le contenu. Chaque cargo est également lié à un port de chargement et à un port de déchargement ainsi qu'à des intervalles de temps durant lesquels le chargement et le déchargement doivent être effectués.

Dès lors, l'opérateur est face à trois problèmes de taille : le premier est d'assigner les cargos aux chimiquiers, le deuxième est de placer ces cargos dans les cuves et le dernier, de trouver la meilleure route à emprunter pour ce bateau.

Premièrement, l'assignation des cargos aux bateaux est un problème combinatoire complexe. En effet, pour m bateaux et n cargos, il y a m^n possibilités différentes. Pour un petit problème avec 10 bateaux et 20 cargos, le nombre de possibilités est déjà de 100.000.000.000.000.000 assignations possibles.

Le problème de placement est lui aussi complexe, vu le grand nombre de combinaisons possibles et le nombre de contraintes. En effet, en plus de savoir dans quel bateau chaque produit va se trouver, l'allocation aux cuves doit être décidée. Cette allocation est contrainte par les produits déjà présents dans le bateau mais également présents dans le futur.

Pour terminer, pour chaque bateau, il faut trouver le chemin entraînant les coûts les plus faibles. Ce problème sans contrainte fait partie de la classe des problèmes *NP-Hard*. En effet, pour un problème comprenant n nous avons $(n - 1)!$ différents chemins possibles. De plus, de nombreuses contraintes qui sont parfois compliquées à modéliser viennent s'ajouter au problème. Par exemple, le fait de savoir si un bateau peut rentrer dans un port dépend de plusieurs facteurs. Les ports possèdent une limite du tirant d'eau autorisé, influencée par la marée. Le tirant d'eau du bateau, quant à lui, est influencé par le chargement mais aussi par la saison. D'autres frais interviennent également. Par exemple, le carburant, qui dépend de la vitesse du bateau de manière quadratique, ainsi une fonction du prix par rapport à la vitesse par mile nautique est : $f(v) = 0,0036v^2 - 0,1015v + 0,8848$, ce qui va, pour différentes vitesses, impliquer différentes combinaisons de ports. Dans les frais interviennent également les frais de ports et de canaux.

Le problème, dans son entièreté, est très compliqué à résoudre. En effet, chaque problème doit être résolu en même temps car les trois problèmes sont hautement interdépendants. Par exemple, un changement d'attribution changera les disponibilités des cuves qui, à leur tour, pourront rendre certains itinéraires impossibles. L'itinéraire va donc changer. De même, le changement de l'itinéraire va modifier la charge entre 2 ports et pourrait rendre le bateau instable après un déchargement...

La difficulté de ce problème rend le calcul d'un optimum global impossible pour des problèmes de taille réelle. Nous allons, dès lors, nous concentrer sur le problème consistant à répartir les produits dans les cuves d'un bateau. Pour se faire, nous allons utiliser les instances générées dans l'article [3].

2.2 Problème d'allocation des cuves

Le problème d'allocation de cuves est introduit dans [3] sous le nom TAP (Tank Allocation Problem). Une instance correspond au trajet d'un chimiquier. Ce chimiquier contient 24 ou 38 cuves et doit suivre un itinéraire prédéfini. Dans chaque port, il peut soit charger des produits, soit en décharger. Le bateau ne doit pas être stable lors du chargement ou du déchargement, mais doit l'être à la sortie du port. Pour chaque instance, le but est de trouver une allocation de cuves possible, voire optimale. Le fait de trouver une solution, même non optimale, assez rapidement est possible car cela permet éventuellement à un opérateur de décider si oui ou non il peut négocier un cargo. Pour ce problème, nous tenterons de minimiser le nombre de cuves à nettoyer au long du voyage. Les contraintes que la solution doit satisfaire sont les suivantes :

1. Chaque cargo doit être alloué à une ou plusieurs cuves ; la quantité du produit ne peut pas dépasser la somme des capacités des cuves allouées.
2. Les cuves peuvent avoir différents attributs (revêtement, gestion de la température...). Les produits ne peuvent être attribués qu'aux cuves compatibles.
3. L'allocation d'un produit à certaines cuves est définitive. Les produits ne peuvent pas changer de cuves.
4. Les cuves ne peuvent contenir qu'un seul type de produit en même temps.
5. Si une cuve est allouée à un produit, un volume minimal doit y être placé pour éviter le ballottage pendant le voyage.
6. Certaines règles sur les produits dangereux empêchent deux produits d'être placés dans des cuves voisines.
7. Certaines règles sur les produits dangereux empêchent deux produits d'être dans le même bateau à un même instant.
8. Certaines règles sur les produits dangereux empêchent deux produits d'être placés dans la même cuve consécutivement sans que la cuve soit nettoyée (ce qui peut coûter cher et prendre beaucoup de temps). Deux produits ne sont pas considérés comme étant consécutifs dans une cuve s'il y a eu un certain nombre de produits (généralement trois) dans cette cuve entre les deux produits incompatibles. Cela signifie que pour chaque cuve, un historique doit être conservé.
9. Pour des raisons de stabilité et de solidité du bateau, certains modèles de chargement doivent être proscrits.

Etant donné que nos instances forcent le trajet, les chargements et les déchargements, la contrainte 7 n'a pas d'influence pour notre problème.

Pour les contraintes 6 et 8, il est nécessaire d'avoir une information sur la compatibilité de deux produits. Pour ce faire, chaque produit est attribué à un type. Le type est représenté par un entier compris entre 0 et 2. Les produits de type 0 sont compatibles avec tous les autres produits. Ceux de type 1 sont incompatibles avec les produits de type 2. Enfin, les produits de type 2 sont incompatibles avec tous les autres produits de type 2 et ceux de type 1.

Le tableau de la figure 2.1 montre la compatibilité de deux produits selon leurs types (un horizontalement et le second verticalement). Les $\bullet$ représentent les types de produits compatibles et les x, les types de produits incompatibles.

Pour la contrainte 2, on utilise également les types des produits et on intègre, en plus, un type pour les cuves. Ce type peut valoir 1 ou 2 et un produit ne peut être placé que dans une cuve avec un type plus grand ou égal au sien.

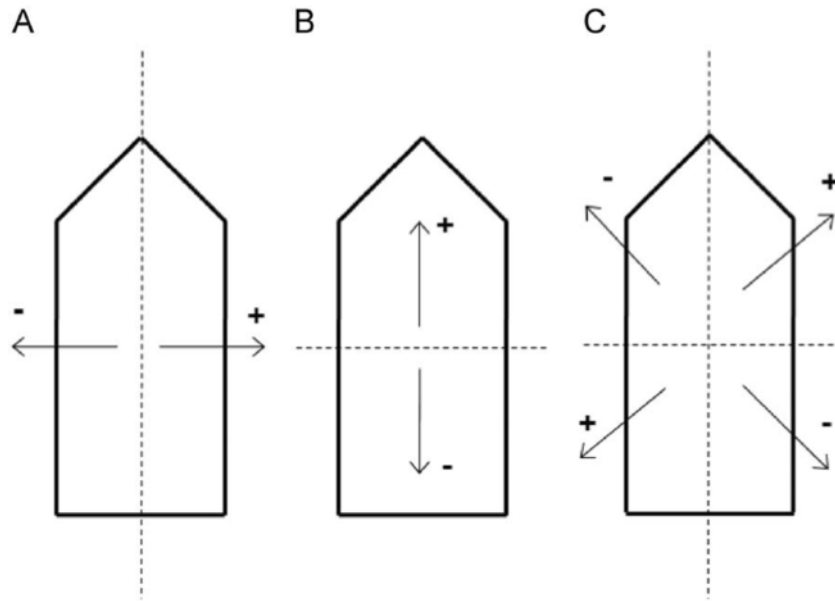
La stabilité du bateau introduite dans la contrainte 9 est gérée par trois moments de forces décrits à la figure 2.2. Chaque cuve possède un facteur par moment. La somme des

FIGURE 2.1: Compatibilité des produits

	0	1	2
0	•	•	•
1	•	•	x
2	•	x	x

masses contenues dans les cuves multipliée par ces facteurs nous donnera trois moments de forces pour le bateau. Ces moments doivent être compris dans des intervalles fournis afin que le bateau soit stable et que sa solidité ne soit pas compromise. Cette contrainte est simplifiée par rapport aux calculs réels qui renverraient des expressions non-linéaires. Cependant, il existe des preuves montrant que la différence due à la simplification est assez faible.

FIGURE 2.2: Moments de force pour représenter la stabilité et la solidité.
©[3]



2.3 Programmation linéaire

2.3.1 Modèle

Les notations et le modèle de programmation linéaire sont également repris de [3]. Certaines notations ont juste été modifiées pour être plus représentatives de la traduction.

– Sets

P set des produits

C set des cuves

P_p^P set des produits en conflit avec le produit p

P_c^C set des produits compatibles avec la cuve c

C_p^P set des cuves compatibles avec le produit p

C_{pqc} set des cuves qui ne peuvent pas être utilisées pour le produit q si le produit p est présent dans la cuve c

- S set des dimensions de stabilité et de solidité
- N_p set des produits présents dans le bateau juste après avoir chargé le produit p
- A_p set des produits qui ont été présents dans le bateau avant d'ajouter le produit p
- Q ensemble de sets ; chaque set correspond aux produits présents dans le bateau entre deux ports (quand les stabilités doivent être respectées)
- R set de produits utilisés pour les sommes
- Indices
 - p, q, r produits
 - c, d cuves
 - s dimensions de stabilité ou solidité
- Variables
 - x_{pc} indicateur pour les allocations ; $x_{pc} = 1$ si et seulement si le produit p est alloué à la cuve c
 - y_{pc} indique la quantité en volume du produit p dans la cuve c
 - z_{pc} indicateur pour les nettoyages ; $z_{pc} = 1$ si et seulement si la cuve c est nettoyée avant d'ajouter le produit p
- Paramètres
 - v_p volume du produit p
 - d_p densité du produit p
 - a_c capacité (en volume) de la cuve c
 - b_c volume minimum dans la cuve c si elle est utilisée
 - m_c^s moment de force pour la dimension s de stabilité ou solidité pour la cuve c
 - m^{s+} borne supérieure pour le moment de force pour la dimension s de stabilité ou de solidité
 - m^{s-} borne inférieure pour le moment de force pour la dimension s de stabilité ou de solidité
 - h_{pqc} nombre de produits nécessaires entre les produits incompatibles p et q dans une même cuve pour ne pas devoir la nettoyer.

La première contrainte garantit que suffisamment de cuves soient utilisées pour contenir l'entière du produit. Elle est contrôlée par ces deux contraintes :

$$y_{pc} \leq a_c x_{pc} (p \in P, c \in C_l^L) \quad (2.1)$$

$$\sum_{c \in C_p^P} y_{pc} = v_p (p \in P) \quad (2.2)$$

L'expression 2.1 garantit que le volume assigné dans une cuve pour un produit ne dépasse jamais la capacité de la cuve. L'expression 2.2 garantit, quant à elle, que le volume total assigné pour un produit est le même que le volume à transporter pour ledit produit.

La deuxième contrainte stipulant que les produits ne peuvent être attribués qu'aux cuves compatibles est également régie par les expressions 2.1 et 2.2. En effet, dans les calculs, ces expressions ne prennent en compte que les cuves compatibles. Cela ne garantit pas que x_{pc} et y_{pc} auront 0 comme valeur mais ces valeurs ne seront pas prises en compte. Par conséquent, les mettre à 0 dans la solution rend celle-ci correcte par rapport à toutes les contraintes, plus particulièrement la deuxième.

La troisième contrainte, portant sur le fait que les produits ne peuvent être changés de cuves, est toujours satisfaite dû au choix de la variable. En effet, l'allocation de la cuve c au

produit p est faite en mettant x_{pc} à 1. Il n'y a, dès lors, pas de question de temps et toute allocation de cuve est définitive.

La quatrième contrainte, spécifiant qu'une cuve ne peut contenir qu'un produit à un même instant, est forcée par l'expression suivante :

$$\sum_{q \in P_c^C \cap N_p} x_{qc} \leq 1 (p \in P, c \in C) \quad (2.3)$$

Pour la cinquième contrainte, permettant d'éviter le ballotement, nous utilisons l'expression :

$$b_c x_{pc} \leq y_{pc} (p \in P, c \in C_l^L) \quad (2.4)$$

Cette expression garantit également que la variable y_{pc} contienne la valeur 0 si une cuve c n'est pas allouée à un produit p compatible.

La sixième contrainte vérifie que deux produits n'étant pas compatibles ne se retrouvent pas dans des cuves voisines. Elle est appliquée par l'expression

$$\sum_{q \in P_p^P \cap N_p} \sum_{d \in C_{pqc}} x_{qd} \leq M_{pc} (1 - x_{pc}) (p \in P, c \in C_p^P) \quad (2.5)$$

Dans cette expression, le coefficient de la partie gauche de l'inégalité compte le nombre de produits incompatibles avec p dans les cuves voisines de c . La partie droite vaut 0 si la cuve c est allouée au produit p et empêche donc la présence de tout produit voisin incompatible. Si la cuve c n'est pas allouée au produit p , la partie de droite vaut un coefficient. Pour que cette contrainte n'exclue aucune solution, ce coefficient doit être supérieur ou égal au nombre de cuves rendues indisponibles pour un produit par l'affectation de la cuve c au produit p . Le coefficient suivant peut donc être utilisé.

$$M_{pc} = \max_q |C_{pqc}| \quad (p \in P, c \in C_p^P)$$

La septième contrainte, quant à elle, empêche certains produits de se trouver simultanément dans le même bateau. Elle n'a, comme indiqué précédemment, aucun intérêt dans ce problème étant donné que la route et les produits sont imposés. Dès lors, nous n'avons aucune influence sur ce qui se trouve dans le bateau à un instant t .

La huitième contrainte a pour but de maintenir la stabilité et la solidité du bateau. Elle consiste simplement à calculer le moment global du bateau pour chaque dimension s et vérifie qu'elle est dans les limites acceptables.

$$m^{s-} \leq \sum_{p \in R} \sum_{c \in C_p^P} m_c^s d_p y_{pc} \leq m^{s+} \quad (2.6)$$

La dernière contrainte, celle qui assure que deux produits incompatibles ne se succèdent pas sans nettoyage dans la même cuve est maintenue par l'expression suivante :

$$\underbrace{h_{pqc}(x_{pc} - z_{pc})}_1 - \underbrace{\sum_{r \in R} (x_{rc} + h_{pqc} z_{rc})}_2 \leq \underbrace{h_{pqc}(1 - x_{qc})}_3 \quad (p \in P, c \in C_p^P, q \in A_p \cap P_p^P, R = A_p \setminus A_q \setminus \{q\}) \quad (2.7)$$

Dans cette expression, q représente les produits incompatibles avec le produit p et qui étaient présents avant le produit p dans le bateau. Cette expression doit donc être satisfaite pour toute cuve et tout couple de produits incompatibles. La partie 1 de l'expression retourne le nombre de produits nécessaires entre les deux produits. La partie 2 compte le nombre de produits qui sont alloués à la cuve c entre les produits p et q et vérifie si cette cuve a été nettoyée pour ces produits. La partie 3 est supérieure à 0 uniquement si la cuve c est assignée au produit p pour que la contrainte soit, le cas échéant, toujours vérifiée.

2.3.2 Implémentation

L'implémentation du modèle de programmation linéaire est faite en utilisant Oscar [5]. Outre le fait qu'elle peut être réalisée en utilisant Scala, un avantage intéressant de l'utilisation d'Oscar pour un modèle de programmation linéaire est le fait que le solveur peut être interchangé en modifiant une seule ligne de code. Les solveurs disponibles sont lpsolve, gurobi et cplex. Le solveur utilisé pour ce mémoire sera lpsolve.

Pour représenter une instance de problème, nous utilisons la classe *TAPInstance*. Pour des raisons de cohérence avec Oscar, le code est écrit en anglais. Les produits sont appelés *Load* et les cuves *Tank*. Afin de faciliter la lisibilité, nous n'allons pas utiliser les mêmes notations que celles présentes dans la section 2.3.1. La classe *TAPInstance* contient les méthodes *Loads*, *Tanks* et *Times*. Ces méthodes renvoient respectivement une séquence des produits, des cuves ou des temps de l'instance. Le temps est géré par un compteur et est incrémenté à chaque action, que ce soit un chargement ou un déchargement. La classe possède également les méthodes *numLoads*, *numTanks*, *numTimes* et *numStabilities*. Ces méthodes renvoient respectivement : le nombre de cuves, le nombre de produits, le nombre d'actions et le nombre de dimensions de stabilité. Cela est utilisé ici uniquement en interne de la classe *TAPInstance* et pour indiquer si oui ou non la stabilité est requise. Cela sera davantage utilisé lors de l'implémentation du solveur à l'aide de la programmation par contraintes. La classe contient également les méthodes *loadsForTank(t)* qui renvoient tous les produits compatibles avec la cuve *t* et la fonction inverse *tanksForLoad(t)*.

L'implémentation du solveur par programmation linéaire prend un argument : *ti* qui est une instance de la classe *TAPInstance* et représente l'instance à résoudre.

```
1 val mip = MIPSolver(LPSolverLib.lp_solve)
2 var x = Array.ofDim[MIPVar](ti.numLoads + 1, ti.numTanks + 1)
3 var y = Array.ofDim[MIPVar](ti.numLoads + 1, ti.numTanks + 1)
4 var z = Array.ofDim[MIPVar](ti.numLoads + 1, ti.numTanks + 1)
5
6 for(l <- ti.Loads ; t <- ti.tanksForLoad(l))
7 {
8   x(l)(t) = new MIPVar(mip, "x"+l+"", "+t, 0 to 1)
9   y(l)(t) = new MIPVar(mip, "y"+l+"", "+t, 0, ti.maximumVolumeInTank(t))
10  z(l)(t) = new MIPVar(mip, "z"+l+"", "+t, 0 to 1)
11 }
```

L'initialisation du solveur se fait à la ligne 1. C'est ici qu'on choisit quel solveur va être utilisé. Ensuite, on crée les variables x, y et z et on les instancie. Le tableau contient toutes les combinaisons de cuves et de produits, mais seules les combinaisons de cuves et les produits compatibles seront initialisés. On remarque qu'on crée un tableau avec un produit et une cuve en plus que ceux présents dans l'instance. Ceci est dû au fait que les produits et les cuves sont numérotés en commençant à 1. Le produit numéroté 0 sera considéré comme l'absence de produit.

Les variables x et z prennent uniquement les valeurs 0 ou 1. La variable y, quant à elle, peut prendre les valeurs comprises entre 0 et la capacité totale de la cuve qu'elle représente, renvoyée par *maximumVolumeInTank(t)*.

La création d'une variable du solveur prend trois ou quatre paramètres. Le premier est le solveur dans lequel cette variable évoluera. Le second est un label assigné à la variable. Le reste est soit l'ensemble de valeurs que la variable peut prendre (lignes 8 et 10), soit des limites dans lesquelles la variable doit se trouver (ligne 9).

```
1 val objective = sum(for(l <- ti.Loads ; t <- ti.tanksForLoad(l)) yield z(l)(t))
2 mip.minimize(objective) subjectTo {
3   //Ajout des contraintes
4 }
```

Ce code permet de lancer la recherche d'une solution. L'objectif à minimiser est le nombre de cuves nécessitant un nettoyage. Nous allons maintenant ajouter les contraintes. La partie

qui suit n'est pas reprise dans les expressions car elle ne découle pas du problème en lui-même, mais bien des instances que nous utilisons. Elle a pour but d'allouer les produits fournis pour créer l'historique des cuves. La fonction *lockedLoads(l)* de la classe *TAPInstance* renvoie toutes les assignations obligatoires pour le produit *l*.

```

1 for(l <- ti.Loads)
2 {
3   if(ti.lockedLoads(l) != null)
4   {
5     for((tank,volume) <- ti.lockedLoads(l))
6     {
7       mip.add(x(l)(tank) == 1)
8       mip.add(y(l)(tank) == volume)
9     }
10  }
11 }
12

```

Pour la première contrainte, nous avons les expressions (2.2) et (2.1). Dans le code qui suit, *loadsVolumes(l)* retourne le volume total du produit *l*.

```

1 for(l <- ti.Loads) mip.add(sum(for(t <- ti.tanksForLoad(l)) yield y(l)(t)) == ti.
  loadsVolumes(l))
2 for(l <- ti.Loads ; t <- ti.tanksForLoad(l)) mip.add(y(l)(t) <= ti.
  maximumVolumeInTank(t) * x(l)(t))

```

Pour l'expression (2.3), nous utilisons le code suivant où *loadsPresentAfterAdding(l)* renvoie tous les produits présents après avoir ajouté le produit *l*.

```

1 for(l <- ti.Loads; t <- ti.tanksForLoad(l))
2   mip.add(sum(
3     for(k <- ti.loadsPresentAfterAdding(l).intersect(ti.loadsForTank(t)))
4       yield x(k)(t)) <= 1)

```

Le code suivant est l'implémentation de l'expression (2.4). Ici *minimumVolumeInTank(t)* renvoie le volume minimal devant être assigné dans la cuve *t* si celle-ci est utilisée.

```

1 for(l <- ti.Loads ; t <- ti.tanksForLoad(l))
2   mip.add(y(l)(t) >= ti.minimumVolumeInTank(t) * x(l)(t))

```

La partie qui suit est utilisée pour implémenter l'expression (2.3). Dans ce code, la méthode *loadsPresentAfterAdding(l)* retourne les produits présents sur le bateau immédiatement après que le produit *l* ait été ajouté.

```

1 for(l <- ti.Loads; t <- ti.tanksForLoad(l))
2   mip.add(sum(
3     for(k <- ti.loadsPresentAfterAdding(l).intersect(ti.loadsForTank(t)))
4       yield x(k)(t)) <= 1)

```

Dans la portion de code qui suit, on établit l'expression (2.5). La méthode *indisponibleTankForLoadIfLoadIn(k,l,t)* renvoie les cuves qu'on ne peut pas utiliser pour mettre le produit *k* si le produit *l* est dans la cuve *t*. La méthode *loadIncompatibleWithLoad(l)*, quant à elle, renvoie les produits incompatibles avec le produit *l*.

```

1 var M = Array.ofDim[Int](ti.numLoads + 1, ti.numTanks + 1)
2 for(l <- ti.Loads ; t <- ti.tanksForLoad(l))
3   M(l)(t) = (for(k <- ti.Loads) yield ti.indisponibleTankForLoadIfLoadIn(k,l,t).size).
4     max
5   for(l <- ti.Loads ; t <- ti.tanksForLoad(l))
6   {
7     mip.add(sum(
8       for(k <- ti.loadIncompatibleWithLoad(l).intersect(ti.loadsPresentAfterAdding(l))
9         yield x(k)(u) <= M(l)(t)*(1-x(l)(t)))
10    )

```


L'expression (2.6) est implémentée par le code suivant dans lequel la méthode *stabilityNeededTimes* renvoie tous les temps nécessitant la stabilité du bateau. *loadsPresentAt(p)* renvoie les produits présents dans le bateau à l'instant p . Les méthodes *momentsMin(s)* et *momentsMax(s)* donnent les limites de stabilité relatives à la dimension s , *momentsTanks(s)(t)* renvoie le facteur de stabilité pour la dimension s et la cuve t et finalement, *density(l)* renvoie la densité du produit l

```

1 for(p <- ti.stabilityNeededTimes; s <- 0 until ti.numStabilities)
2 {
3   mip.add(
4     sum(
5       for(l <- ti.loadsPresentAt(p); t <- ti.tanksForLoad(l))
6         yield (y(l)(t)*(ti.momentsTanks(s)(t)* ti.density(l)))
7         >= (ti.momentsMin(s) )
8     )
9   )
10  mip.add(
11    sum(
12      for(l <- ti.loadsPresentAt(p); t <- ti.tanksForLoad(l))
13        yield (y(l)(t)* (ti.momentsTanks(s)(t)* ti.density(l)))
14        <= (ti.momentsMax(s) )
15    )
16  }

```

Pour la dernière expression (2.3.1), la méthode *loadIncompatibleWithLoad(l)* retourne les produits incompatibles avec le produit l , la méthode *loadsPrior(l)* retourne tous les produits qui se sont trouvés dans le bateau avant le produit l , et la méthode *loadsPrior* retourne le nombre de produits nécessaires entre deux produits pour que la cuve ne nécessite pas de nettoyage.

```

1 for(l <- ti.Loads ;
2   t <- ti.tanksForLoad(l);
3   k <- ti.loadsPrior(l).intersect(ti.loadIncompatibleWithLoad(l)).intersect(ti.
4     loadsForTank(t))
5 {
6   val R = ti.loadsPrior(l).diff(ti.loadsPrior(k)).filter(_ != k).intersect(ti.
7     loadsForTank(t))
8   mip.add(ti.loadsForClean * (x(l)(t) - z(l)(t)) - sum(for(j<-R)
9     yield x(j)(t) + ti.loadsForClean * z(j)(t)) <= ti.loadsForClean*(1-x(k)(t)))

```

2.4 Programmation par contraintes

Les contraintes sont, en général, plus faciles à exprimer avec la programmation par contraintes qu'avec la programmation linéaire. Cela permet d'avoir des modèles plus flexibles, car de nouvelles contraintes sont ajoutables assez simplement. Une conséquence de cela est également un gain en confiance de l'utilisateur pour le modèle. Outre la difficulté de la réalisation d'un système de routage de chimiquier entièrement automatique, un frein à l'utilisation de systèmes d'aide à la décision est le manque de confiance en ceux-ci par les utilisateurs. Le fait d'avoir des contraintes assez faciles à comprendre, même pour quelqu'un n'ayant pas ou peu de formation en mathématiques - ce qui est en général le cas des utilisateurs de ces systèmes - est un grand avantage. En effet, on a toujours moins peur de ce qu'on peut comprendre. La même réflexion peut être faite sur la stratégie adoptée pour résoudre le problème : alors que la programmation linéaire utilise des techniques mathématiques avancées, la programmation par contraintes consiste basiquement à essayer toutes les solutions possibles. En pratique, bien sûr, un nombre, le plus grand possible, de solutions n'est pas considéré car exclu de la recherche. Cependant, ceci est bien plus facile à expliquer et à comprendre que les algorithmes utilisés dans les solveurs de programmation linéaire.

Le modèle, bien que représentant le même problème, est assez différent de celui décrit à la section 2.3.1, entre autres, parce que les variables de décision sont différentes et donc, toutes

les contraintes le sont aussi.

Trois tableaux de variables de décision sont utilisés pour modéliser le problème :

tanksContent(time)(tank) contient pour tout temps *time*, le produit contenu dans la cuve *tank*.

assignedVolume(time)(load) contient pour tout temps *time*, la somme des capacités des cuves assignées au produit *load*.

cleanTank(tank)(load) vaut 1 si et seulement si la cuve *tank* doit être nettoyée avant de charger le produit *load*.

2.4.1 Contraintes

La première contrainte du problème spécifiant que la capacité des cuves allouées à un produit soit suffisante pour contenir l'entière du produit est garantie, en partie, par la définition du problème. En effet, *assignedVolume(time)(load)* a un domaine qui commence au volume nécessaire pour le produit et on ne peut donc lui en assigner moins. L'allocation des cuves aux produits est réalisée par une contrainte *BinPacking* qui est expliquée plus en détail à la section 2.4.1.

Pour la seconde contrainte, stipulant que les produits ne peuvent aller que dans des cuves compatibles, c'est le domaine de la variable *tanksContent(time)(tank)* qui s'en charge. Il suffit, en effet, que seuls les produits compatibles soient présents dans le domaine pour que la contrainte soit respectée.

La troisième contrainte, quant à elle, est garantie par une contrainte développée pour ce mémoire : *ShareValue*. Cette contrainte est expliquée à la section 2.4.1.

La quatrième contrainte, assurant qu'une cuve ne contient qu'un seul type de produit à la fois est toujours vérifiée par définition des variables de décision. En effet, la variable *tanksContent(time)(tank)* ne contient qu'un seul type de produit à tout instant.

Les contraintes cinq et neuf ne sont pas considérées pour le moment. Ces contraintes portent sur les volumes assignés dans les cuves et sur la stabilité du bateau. Ces allocations n'étant pas entières, la programmation par contraintes n'est pas l'outil le plus adapté pour les résoudre. Par conséquent, nous nous occupons tout d'abord d'un problème où ces contraintes sont relaxées et nous envisagerons une méthode pour les prendre en compte à la section 2.4.4.

La sixième contrainte régulant l'association de produits différents à des cuves voisines est implémentée par une contrainte utilisant une table décrite à la section 2.4.1.

La septième contrainte interdisant la présence de certains produits simultanément dans le bateau n'est, à nouveau, pas pertinente.

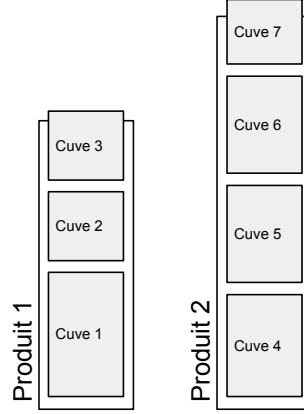
La dernière contrainte, la huitième, s'occupant de l'historique et du nettoyage de la cuve est expliquée à la section 2.4.1.

BinPacking

La contrainte *BinPacking* peut être vue comme une contrainte allouant des objets dans des boîtes. Chaque objet a un poids et à chaque boîte correspond une capacité. La somme des poids des objets alloués à une boîte doit être égale à sa capacité. Dans la pratique, la contrainte est définie par :

$$\text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m])$$

FIGURE 2.3: Allocation de cuves aux produits



où n objets sont placés dans m boîtes. Les variables $[X_1, \dots, X_n]$ représentent le placement de chaque objet dans une boîte. Par exemple, $X_i = j$ signifie que l'objet i est placé dans la boîte j , $[w_1, \dots, w_n]$ sont des constantes qui représentent le poids de chaque objet et, pour finir, les variables $[L_1, \dots, L_m]$ contiennent la somme des poids des objets alloués à chaque boîte. La relation est la suivante :

$$\forall i \in [1..m] : \sum_{i|X_i=j} w_i = L_j$$

L'utilisation de cette contrainte pour notre problème peut sembler contre-intuitive car les objets seront les cuves et les boîtes, les produits. En effet, à chaque produit sera assigné une ou plusieurs cuves de telle sorte que la somme des capacités des cuves assignées soit plus grande ou égale au volume de produits à assigner. Ceci est décrit à la figure 2.3 : on peut y voir deux produits ainsi que les cuves qui leur sont assignées. Nous utilisons également un produit fictif, le produit 0, dans lequel on place les cuves qui sont inutilisées.

Nous utiliserons trois implémentations de cette contrainte et analyserons les conséquences que les différentes techniques ont sur les performances des résolutions. La première technique sera d'utiliser la méthode décrite dans [8]. La technique suivante sera d'ajouter une contrainte redondante à cette contrainte, celle décrite dans [7]. Cette contrainte ajoute une réflexion sur le nombre d'éléments allant dans chaque boîte. Ici, le nombre de cuves allouées à chaque produit. La dernière technique sera d'utiliser la contrainte décrite dans [8] et une version de la contrainte dans [8], modifiée pour améliorer l'élagage de l'arbre de recherche. Cette contrainte est l'objet du chapitre suivant.

ShareValue

La contrainte *ShareValue* a été créée pour ce problème. Elle permet de garantir qu'un ensemble de variables partage une valeur précise. Pour que cette contrainte soit satisfaite, les variables présentes dans le set doivent être égales uniquement si cette valeur est assignée au moins à une variable du set. Pour la contrainte :

$$ShareValue([X_1, \dots, X_n], v)$$

Les n variables X_i doivent soit être toutes égales, soit aucune ne doit l'être ; c'est-à-dire :

$$\forall i \in [1..n] X_i = v \vee \forall i \in [1..n] X_i \neq v$$

Cette contrainte est implémentée de manière très simple. Lorsque la valeur v est enlevée du domaine d'une des variables $X_1, \dots, X_n$, elle est enlevée du domaine de toutes les variables.

Lorsqu'une variable du set est assignée, si la valeur assignée est v , elle est assignée à toutes les variables du set. Si ce n'est pas v , v est supprimé de tous les domaines des variables comprises dans le set.

Cette contrainte est utilisée pour garantir que les produits restent dans la même cuve durant tout leur voyage. Pour ce faire, pour chaque cuve et chaque produit compatible avec cette cuve, est ajouté la contrainte *ShareValue* où l'ensemble des variables est l'ensemble des variables représentant la cuve à tous les instants pour lesquels le produit est dans le bateau, et la valeur à partager est l'index du produit.

Table

Pour toute paire de cuves voisines, on ajoute une contrainte *table* qui consiste en une table de toutes les combinaisons acceptables que peuvent prendre les deux cuves. Pour que la contrainte soit satisfaite, la combinaison des produits contenus dans les deux cuves doit se trouver dans la table.

Historique et nettoyage

Le fonctionnement de la contrainte gérant l'historique des cuves est assez similaire à celui du modèle de programmation linéaire.

Pour chaque paire de produits incompatibles et chaque cuve compatible avec les deux produits, nous allons vérifier que nous avons bien une de ces quatre possibilités :

1. Le premier produit n'est pas assigné dans la cuve.
2. Le second produit n'est pas assigné dans la cuve.
3. Un nombre minimum défini de produits a été placé dans la cuve entre les deux produits incompatibles.
4. La cuve a été nettoyée au moins une fois entre les deux produits.

Pour vérifier qu'un produit a été placé dans la cuve, il est uniquement nécessaire de vérifier la variable *tanksContent* au moment où le produit est chargé dans le bateau. En effet, la contrainte *ShareValue* assure que, si le produit est assigné à la cuve, il le sera au moment de son chargement.

2.4.2 Recherche

La nombre de nettoyages de cuves étant à minimiser, nous commencerons par mettre les variables représentant le nettoyage d'une cuve à la valeur signifiant que le nettoyage n'est pas nécessaire.

Une fois que cela sera fait, nous devrons assigner des valeurs aux variables *tanksContent*. Il n'est pas nécessaire d'assigner les variables *assignedVolume* cars elles découlent directement des valeurs des différentes variables *tanksContent*.

Nous utiliserons la technique qui assigne en priorité une valeur à la variable qui est la plus à même de créer un échec, c'est-à-dire la variable avec le plus petit domaine. Nous chercherons après la variable qui, quant à elle, a la plus grande probabilité de mener à un optimum. Nous prendrons, pour ce faire, le produit qui a la plus grande quantité restant à placer.

Une fois que nous aurons trouvé la variable à assigner et la valeur qui lui sera donnée, deux branches seront créées, une avec cette valeur forcée, et l'autre avec cette valeur interdite.

2.4.3 Implémentation

La première étape dans l'implémentation est d'initialiser le solveur et de définir les variables de décision.

```

1 val cp = CPSolver()
2
3 val tanksContent = Array.tabulate(ti.numTimes, ti.numTanks + 1)((time, tank) =>
4   CPVarInt(cp, 0 to ti.loadsForTank(tank).intersect(ti.loadsPresentAt(time))))
5 val assignedVolume = Array.tabulate(ti.numTimes, ti.numLoads + 1)(
6   (time, load) => CPVarInt(cp, possibleVolumesAssignedAtFor(time, load)))
7 val cleanTank = Array.tabulate(ti.numTanks + 1, ti.numLoads + 1)(
8   (tank, load) => CPVarInt(cp, 0 to 1))

```

Ici, les variables de décision sont initialisées. Le domaine de *tanksContent*(*time*)(*tank*) contient tous les produits compatibles avec la cuve *tank* et présents dans le bateau au moment *time*. Chaque variable *assignedVolume*(*time*)(*load*), initialisée si le produit *load* est présent au moment *time*, possède un domaine allant du volume nécessaire pour le produit *load* à la somme des capacités de toutes les cuves. Si le produit n'est pas dans le bateau à cet instant, il est fixé à 0 et, dans le cas où le produit est le produit 0 signifiant que la cuve est vide, la variable a un domaine allant de 0 à la capacité totale du bateau.

Pour ajouter les contraintes et paramétrer la recherche, on utilise la structure suivante qui permet de minimiser le nombre de cuves à nettoyer :

```

1 cp.minimize(sum(cleanTank.flatten)) subjectTo{
2   // Ajout des contraintes
3 } exploration {
4   // Stratégie de recherche
5 } run()

```

Comme pour le modèle de programmation linéaire, on doit assigner les produits forcés, ceci est fait en ajoutant la contrainte :

```

1 for(l <- ti.Loads)
2 {
3   if(ti.lockedLoads(l) != null)
4   {
5     for((tank, volume) <- ti.lockedLoads(l))
6     {
7       mip.add(tanksContent(ti.loadTime(l))(tank) == 1)
8     }
9   }
10 }

```

Nous ajoutons ensuite la contrainte *BinPacking*. Comme indiqué, cette contrainte sera ajoutée de trois manières suivantes. La première contrainte sera présente à tous les coups :

```

1 for(time <- ti.Times)
2 {
3   cp.add(new BinPacking(tanksContent(time),
4     ti.maximumVolumeInTank.map(_ toInt),
5     assignedVolume(time)), Strong)
6 }

```

Ensuite, nous avons la contrainte décrite dans [7]. Pour cette contrainte, nous avons besoin d'autres variables qui représentent le nombre de cuves pour chaque produit.

```

1 cp.add(new BinPackingFlow(tanksContent(time),
2   ti.maximumVolumeInTank.map(_ toInt),
3   assignedVolume(time),
4   Array.tabulate(ti.numLoads + 1)((load) => CPVarInt(cp, 0 to ti.numTanks
5     ) )))

```

La dernière version de *BinPacking* prendra également la contrainte issue de [8] et la seconde contrainte sera ajoutée de la même manière que la contrainte précédente, mais avec le nom *BinPackingFlowExtended* au lieu de *BinPackingFlow*.

Pour les contraintes *ShareValue*, l'ensemble des variables représentant les cuves compatibles avec un produit à tous les temps compris entre le chargement et le déchargement dudit produit sont récupérées et la contrainte *ShareValue* les force à partager l'index du produit.

```

1 for (load <- ti.Loads; tank <- ti.tanksForLoad(load))
2   cp.add(new ShareValue(tanksContent.slice(ti.loadTime(load), ti.unloadTime(load))
   .map(_(tank)), load))

```

Pour contraindre le contenu des cuves voisines, on impose une table de produits compatibles pour toutes les cuves à tout instant.

```

1 for (time <- ti.Times)
2   for (t1 <- ti.Tanks)
3     for (t2 <- ti.neighbors(t1))
4       cp.add(table(tanksContent(time)(t1), tanksContent(time)(t2), compatibles))

```

Pour la dernière contrainte, pour toute paire de variables incompatibles et pour chacun des quatre cas décrits dans la section 2.4.1, une variable de type *CPVarBool* vaut *vrai* si et seulement si on se trouve dans ce cas. Une contrainte force qu'au minimum une de ces quatre variables soit assignée à la valeur *vrai*. Dans le code suivant, les méthodes *areLoadsIncompatible* et *loadAndTankCompatible* de la classe *TAPInstance* sont utilisées pour tester la compatibilité respectivement entre deux produits et entre une cuve et un produit. La méthode *isLoadInTank(l,t)*, quant à elle, vérifie si le produit *l* est placée dans la cuve *t*.

```

1 for (l1 <- ti.Loads)
2   for (l2 <- ti.loadsPrior(l1))
3     .filter(l=>ti.areLoadsIncompatible(l1, l))
4     for (tank <- ti.Tanks.filter(ti.loadAndTankCompatible(l1, _))
5       .filter(ti.loadAndTankCompatible(l2, _)))
6       {
7
8
9         val loadsBetween = ti.Loads
10          .filter(l=>(ti.unloadTime(l) < ti.loadTime(l1)
11                    && ti.loadTime(l) > ti.unloadTime(l2)))
12
13         val cleanVars = (l1 :: loadsBetween.toList).map(cleanTank(tank)(_))
14         val cleaned = sum(cleanVars) >= 1
15
16
17         val enoughLoads : CPVarBool=
18           if (loadsBetween.count(ti.areLoadsCompatible(l1, _))
19              >= ti.loadsForClean
20              (sum(loadsBetween.map(l=>isLoadInTank(l, tank)))
21               >= ti.loadsForClean))
22         else
23           CPVarBool(cp, false)
24         cp.add(!isLoadInTank(l1, tank) || !isLoadInTank(l2, tank) || cleaned ||
25           enoughLoads)
26       }

```

L'implémentation de la stratégie de recherche ci-dessous nécessite la méthode *volumeAllocated(l)* qui calcule la somme des capacités des cuves déjà allouées au produit *l*. La méthode *selectMin* quant à elle va sélectionner le plus petit élément d'une liste répondant à une contrainte. Pour ce faire, elle prendra trois arguments, le premier est la liste dans laquelle elle doit trouver l'élément minimal. Le second est la condition que les valeurs doivent vérifier pour pouvoir être sélectionnées. Le troisième est une méthode qui, pour tout élément de la liste passée en premier argument, renvoie un objet appartenant à une classe définissant des méthodes permettant de trier ses instances. Ces objets permettent donc de trier la liste.

```

1 val allTanksContent = tanksContent.flatten
2 val allCleanTank = cleanTank.flatten
3 cp.deterministicBinaryFirstFail(allCleanTank, x =>
4   selectMin(0 to 1)
5     (a => x.hasValue(a))
6     (a=>a).get)
7 cp.deterministicBinaryFirstFail(allTanksContent, x =>
8   selectMin(0 to ti.numLoads)

```

```

9 | (l => x.hasValue(l))
10 | (l=>ti.loadsVolumes(l) - volumeAllocated(l)).get)

```

2.4.4 Allocation des volumes

L'allocation des volumes pour chaque cuve n'est pas décidée avec la programmation par contraintes. Elle peut être réalisée par une résolution par programmation linéaire une fois que l'allocation des cuves aux produits est faite.

Le sous-problème par programmation linéaire est une partie du problème général expliqué dans la section 2.3.1, qui a été légèrement modifié pour prendre en compte le fait que les cuves sont déjà assignées.

```

1 | def tanksAssignedTo(load: Int) = tanksContent(ti.loadTime(load))
2 |                               .zipWithIndex.filter(_._1 == load)
3 |                               .map(_._2)
4 |
5 |
6 | val mip = MIPSolver(LPSolverLib.lp_solve)
7 |
8 | var y = Array.ofDim[MIPVar](ti.numLoads + 1, ti.numTanks + 1)
9 |
10 | for(l <- ti.Loads ; t <- tanksAssignedTo(l))
11 | {
12 |   y(l)(t) = new MIPVar(mip, "y"+l+" "+t, ti.minimumVolumeInTank(t), ti.
13 |     maximumVolumeInTank(t))
14 | }
15 |
16 | mip.maximize(0) subjectTo {
17 |   for(l <- ti.Loads) mip.add(oscar.algebra.sum(for(t <- tanksAssignedTo(l)) yield y(l)
18 |     )(t)) == ti.loadsVolumes(l))
19 |   for(p <- ti.stabilityNeededTimes; s <- 0 until ti.numStabilities)
20 |   {
21 |     mip.add(
22 |       oscar.algebra.sum(
23 |         for(l <- ti.loadsPresentAt(p);
24 |           t <- tanksAssignedTo(l))
25 |         yield (y(l)(t)*(ti.momentsTanks(s)(t)* ti.density(l))))
26 |       )
27 |     )
28 |     mip.add(
29 |       oscar.algebra.sum(
30 |         for(l <- ti.loadsPresentAt(p);
31 |           t <- tanksAssignedTo(l))
32 |         yield (y(l)(t)*(ti.momentsTanks(s)(t)* ti.density(l))))
33 |       )
34 |     )
35 |   }
36 | }

```

2.5 Expérimentation

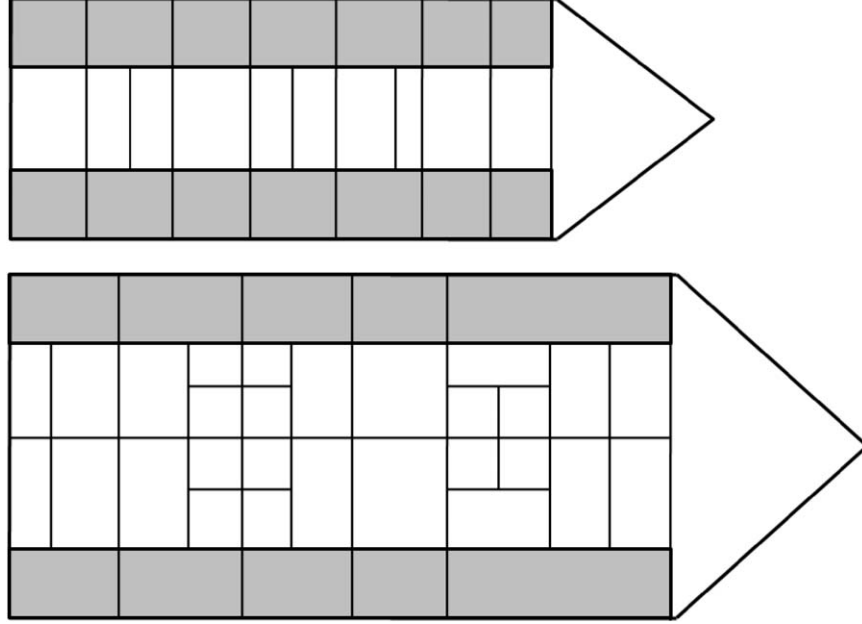
2.5.1 Génération des instances de test.

Pour tester les modèles, les instances générées pour [3] ont été utilisées. Les instances sont générées aléatoirement avec le respect de cinq paramètres. Ces paramètres sont indiqués sous forme de labels dans les noms des fichiers et sont les suivants :

- Il y a deux bateaux différents illustrés à la figure 2.4. Ces bateaux sont de tailles réelles. Le premier contient 24 cuves; c'est un bateau relativement petit qui peut transporter $10000m^3$ de produits pour 8000 tonnes de port en lourd. Le second est plus grand, il est composé de 38 cuves et a une capacité totale de $45000m^3$ de produits pour 39000 tonnes de port en lourd. Ils contiennent deux types de cuves comme cela est représenté

à la figure 2.4. Les premières, représentées en blanc, possèdent un recouvrement d’acier inoxydable (cuve de type 1). Les autres, représentées en gris clair sur le schéma possèdent un recouvrement de silicate zinc (cuve de type 2). Les conditions de solidité et de stabilité du bateau sont calculées selon la configuration des cuves sur le bateau. Les labels sont T24 ou T38 en fonction des bateaux.

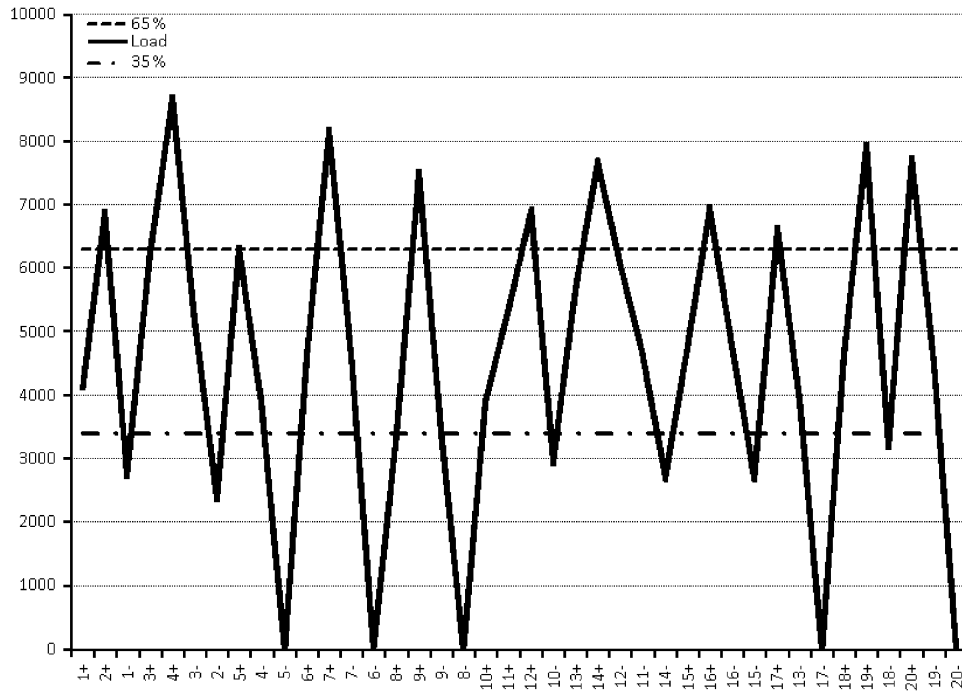
FIGURE 2.4: Bateaux avec 24 et 28 cuves.
©[3]



- Les instances peuvent porter sur 20, 30 ou 40 produits à transporter. Les 10 premiers de ces produits sont déjà alloués aux cuves et servent à allouer un historique pour ces cuves. Les labels utilisés sont L20, L30 et L40
- Les distributions de probabilités pour chaque type. Trois labels sont utilisés : C1 (0,6;0,4;0), C2 (0,5;0,4,0,1) et C3 (0,3;0,4;0,3). C1 représente le cas où 60% des produits sont sensés être de type 1 et 40% des produits sont sensés être de type 2.
- Les capacités maximum et minimum de l’utilisation des cuves avant le chargement et le déchargement ont les labels F1 (0,65;0,35), F2 (0,75;0,25) et F3 (0,85;0,15). Cela signifie, par exemple, que dans une route générée à partir de F1, le bateau va commencer par se charger et, lorsque la somme des quantités de produits dépasse 65% des capacités totales du bateau, il va commencer à se décharger jusqu’à ce que les produits ne remplissent que moins de 35% de la capacité totale du bateau pour ensuite recommencer à se charger. Ces limites ont pour but de varier les séquences de ports chargeant et de ports déchargeant des produits. La figure 2.5 illustre pour l’instance *TAP_T24L20C3F1S3_01* qui utilise, comme indiqué dans le nom, les paramètres de F1, donc (0,65;0,35), pour générer l’ordre des ports et la capacité du bateau à chaque chargement/déchargement.
- La distribution des volumes des produits. Les labels sont S3 (1000–5000 m^3), S6 (3000–9000 m^3) et S12 (8000 – 16000 m^3). Pour S3, cela signifie que les tailles des produits suivent une distribution uniforme entre 1000 et 5000 m^3

Les instances sont générées aléatoirement pour chaque combinaison de paramètres, exceptés pour les combinaisons contenant T24 et S12 car le bateau est trop petit pour les produits considérés, et T38 avec S3 pour les raisons inverses. Pour chaque combinaison de paramètres, 5 instances sont générées. Il y a 144 combinaisons possibles et donc, 720 instances. Chaque

FIGURE 2.5: Évolution de la charge du bateau le long du trajet pour l'instance *TAP_T24L20C3F1S3_01*.
©[3]



instance a été vérifiée comme étant solvable.

Résoudre l'instance revient à allouer chaque produit à plusieurs cuves et à définir quel volume de produit mettre dans chaque cuve.

Les figures 2.6 et 2.7 contiennent le fichier décrivant l'instance *TAP_T24L20C3F1S3_01*. Celle-ci a été traduite pour correspondre aux précédentes annotations. Certains commentaires ont également été retirés car ils étaient incorrects. Il y a une autre erreur présente dans les données : certaines assignations verrouillées de certaines instances violaient les conditions de stabilité. Celles-ci ne doivent être vérifiées que pour les assignations qui sont faites par la résolution du problème et non pas pour celles obligées par les instances. Ces erreurs ont été reportées et confirmées par Lars Magnus Hvattum, auteur de [3].

Ligne 4 Le nombre de produits, de cuves et de dimensions de stabilité.

Ligne 7 Les volumes des différents produits.

Ligne 10 Les masses des différents produits utilisées pour calculer les densités des produits.

Ligne 13 Les volumes minimums nécessaires pour chacune des cuves si elles sont utilisées.

Ligne 16 Les capacités en volume de chaque cuve.

Ligne 19 Les moments proportionnels à la masse de produits contenue de chaque cuve.

Ligne 22 Les types des produits, utilisés pour la compatibilité entre les produits.

Ligne 25 Les types des cuves, utilisés avec les types de produits définis pour les cuves et les produits compatibles.

Lignes 28-51 Les relations de voisinage entre les cuves, utilisées avec les types de produits pour définir les paires d'allocations de cuves et des produits incompatibles.

Ligne 53 Le nombre de produits nécessaires dans une cuve entre deux produits incompatibles pour éviter de nettoyer celle-ci.

Lignes 56-95 Le voyage du bateau. La première colonne représente l'action, c'est-à-dire le chargement ou déchargement d'un produit. Par exemple, le chargement du produit 1 est noté +1 et le déchargement, -1. La deuxième colonne vaut 1 si la stabilité du bateau doit être vérifiée après l'action. La troisième colonne vaut 1 si les assignations aux cuves sont forcées. Ces actions sont là pour créer un historique des cuves. Les derniers éléments représentent les assignations : ils sont de la forme x :y où x est le numéro d'une cuve dans laquelle placer le produit et y est la quantité à y mettre.

FIGURE 2.6: Première partie de l'instance *TAP_T24L20C3F1S3_01*

```

1 # TAP instance
2 #
3 # |P|, |C|, |S|
4 20 24 1
5 #
6 # v
7 4090 2773 3500 2392 3888 4624 3520 3437 4060 3918 1396 1579 2704 4268
8 1981 1898 4632 1335 4000 2605
9 #
10 # w
11 4466 2512 3313 2136 3673 4591 3595 2983 4656 3615 1638 1610 3033 5049
12 1850 1918 3853 1216 3592 2609
13 #
14 # b
15 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
16 #
17 # c
18 314 274 278 278 278 271 320 896 471 353 824 353 471 648 177 839 638 314 274 278 278
19 278 271 320
20 #
21 # m+ m- m
22 -7000 7000 -7 -7 -7 -7 -7 -7 -7 0 0 0 0 0 0 0 0 0 7 7 7 7 7 7
23 #
24 # type de chaque produit
25 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 1 1 0 1
26 #
27 # type de chaque cuve
28 2 2 2 2 2 2 1 1 1 1 1 1 1 2 2 2 2 2 2 2
29 #
30 # liste des voisins
31 1 8 2
32 2 9 10 3 1
33 3 11 4 2
34 4 12 13 5 3
35 5 14 15 6 4
36 6 16 7 5
37 7 17 6
38 8 1 18 9
39 9 2 19 10 8
40 10 2 19 11 9
41 11 3 20 12 10
42 12 4 21 13 11
43 13 4 21 14 12
44 14 5 22 15 13
45 15 5 22 16 14
46 16 6 23 17 15
47 17 7 24 16
48 18 8 19
49 19 9 10 20 18
50 20 11 21 19
51 21 12 13 22 20
52 22 14 15 23 21
53 23 16 24 22
54 24 17 23
55 ## h
56 3
57 #

```

FIGURE 2.7: Seconde partie de l'instance *TAP_T24L20C3F1S3_01*

```
55 # (action , stabilité requise , verrouillé , cuves et volumes assignés)
56 +1 1 1 16:839 7:320 8:896 17:638 2:274 15:177 14:648 18:298
57 +2 1 1 19:274 20:278 9:471 23:271 4:278 3:278 13:471 12:353 10:99
58 -1 1 1
59 +3 1 1 24:320 17:638 15:177 6:271 11:824 22:278 8:896 1:96
60 +4 1 1 7:320 5:278 14:648 21:278 18:314 16:554
61 -3 1 1
62 -2 1 1
63 +5 1 1 9:471 19:274 1:314 2:274 8:896 6:271 12:353 4:278 15:177 3:278 24:302
64 -4 1 1
65 -5 0 1
66 +6 1 1 22:278 20:278 2:274 15:177 18:314 10:353 4:278 9:471 17:638 19:274
14:648 21:278 6:271 13:92
67 +7 1 1 8:896 7:320 3:278 16:839 5:278 23:271 11:638
68 -7 1 1
69 -6 0 1
70 +8 1 1 1:314 12:353 5:278 24:320 20:278 18:314 19:274 15:177 10:353 4:278
3:278 6:220
71 +9 1 1 8:896 16:839 7:320 22:278 14:648 17:638 21:278 2:163
72 -9 1 1
73 -8 0 1
74 +10 1 1 13:471 6:271 20:278 21:278 12:353 16:839 24:320 23:271 22:278 19:274
18:285
75 +11 1 0
76 +12 1 0
77 -10 1 0
78 +13 1 0
79 -12 1 0
80 -11 1 0
81 +14 1 0
82 -14 1 0
83 +15 1 0
84 +16 1 0
85 -16 1 0
86 -15 1 0
87 +17 1 0
88 -13 1 0
89 -17 0 0
90 +18 1 0
91 +19 1 0
92 +20 1 0
93 -18 1 0
94 -19 1 0
95 -20 0 0
96 #
```

2.5.2 Résolution et comparaison de solution par instance

Pour résoudre une instance du problème avec un ou plusieurs solveurs, la classe *TAPRunner* est utilisée. Une fois lancée, elle permet de choisir une instance parmi une liste et un ou plusieurs des quatre différents solveurs :

1. Le solveur utilisant la programmation linéaire
2. Le solveur avec la contrainte BinPacking de [8]
3. Le solveur avec la contrainte BinPacking de [8] et de [7]
4. Le solveur avec la contrainte BinPacking de [8] et du chapitre suivant.

Une fois les résolutions effectuées, il nous est proposé de visualiser les résultats. Un exemple de cette visualisation pour les solveurs 1 et 4 est disponible à la figure . Les flèches disponibles dans la barre d'outils permettent de se déplacer dans le temps pour voir l'état du bateau à différents instants pour chaque solution.

2.5.3 Benchmarks

Lors de la création d'une nouvelle contrainte ou de la modélisation d'un problème, il est souvent intéressant de pouvoir avoir des informations concernant l'efficacité de différentes

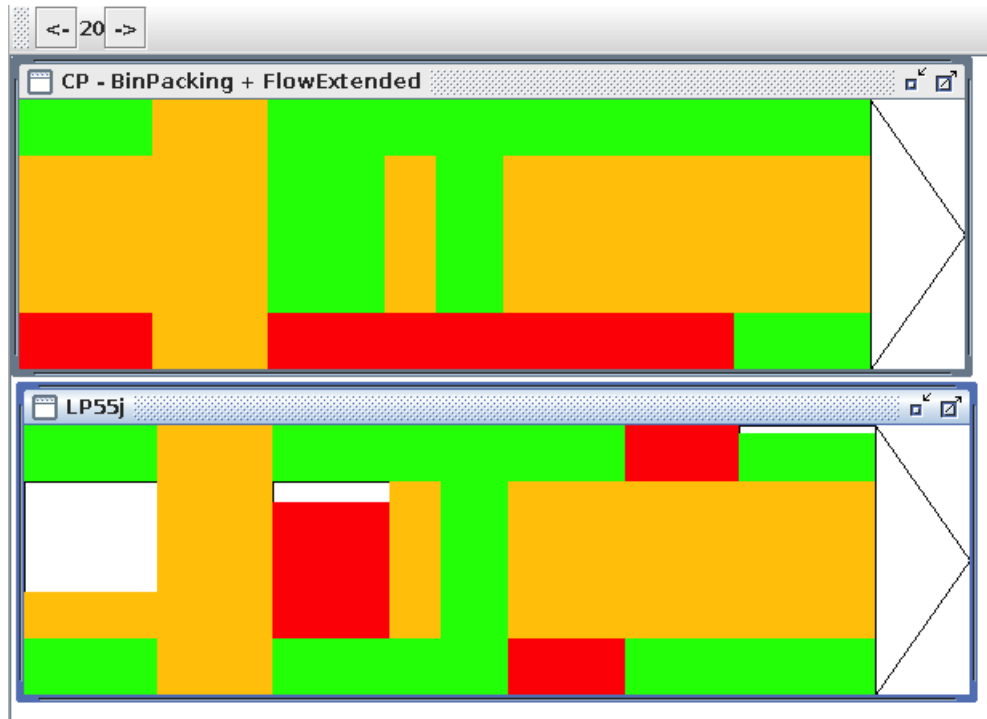


FIGURE 2.8: Comparaison des solutions pour les solveurs 1 et 4 pour l'instance *TAP_T24L20C3F1S3_02*

techniques ou configurations, d'une contrainte redondante, ...

Le rôle de la librairie *benchmark*, écrite dans le cadre de ce mémoire, est de faciliter l'écriture de tests de performances en automatisant les tests et en gérant les résultats. La librairie permet d'effectuer les tests, de sauver les résultats et, ensuite, de les réimporter pour les compléter ou les analyser. Ils peuvent également être exportés dans différents formats pour être facilement récupérés dans un logiciel tel excel ou pour être publié en latex.

Un benchmark est composé de 4 types d'informations différents.

Profils Ils représentent les profils d'une série de tests. Un profil contient typiquement des attributs utilisés pour générer aléatoirement des problèmes ou des paramètres à activer ou à désactiver certaines contraintes. La librairie permet de représenter sur un graphique les résultats par rapport à un attribut du profil. Cela permet de voir facilement l'influence que cet attribut a sur les résultats.

Résultats contient les attributs représentant les résultats d'un test. La librairie calcule, pour chaque attribut et pour chaque paire de profils et de méthodes, le minimum, le maximum et le total des valeurs.

Méthodes Une méthode décrit le moyen utilisé pour résoudre une instance d'un problème. Différentes méthodes peuvent, par exemple, être différents ensembles de contraintes, ou bien une résolution par contraintes et une par programmation linéaire,... La librairie permet, grâce à des graphiques, d'avoir une représentation rapide des performances de différentes méthodes.

Entrée L'environnement d'un test est représenté par un profil, une méthode ainsi qu'une entrée. Cette entrée est, par exemple, le nom d'un fichier contenant une instance pour un test, ou un nombre pour initialiser un générateur de nombres aléatoires...

Un benchmark peut être vu comme une fonction à trois paramètres : un profil, une méthode et une entrée retournant un résultat.

Ces 4 types d'informations sont représentés de la même manière, à l'aide d'un objet contenant la structure et un objet contenant l'information. Pour chaque benchmark et pour chaque type d'information, il y a un objet contenant la structure de cette information. Cet objet correspond à une liste de clés et à chaque clé sont associés un index et un type. Ensuite, pour chaque 'instance' de cette information (par exemple un profil), il y a un objet contenant les valeurs associées aux clés contenues dans la structure. Chaque valeur est de type *BenchValue*. Quatre types sont déjà définis (*BenchString*, *BenchInt*, *BenchDouble* et *BenchBoolean*), mais des sous-classes de *BenchValue* peuvent être créées. Existente également des conversions implicites des types scala *String*, *Int*, *Double* et *Boolean* vers leurs sous-types de *BenchValue* correspondants. En plus de cela, il existe deux objets particuliers :

NullVal représente l'absence de valeur. Il peut être utilisé par l'utilisateur pour indiquer un attribut non pertinent, ou indiquer que le résultat n'est pas encore calculé.

MultiVal est utilisé, par exemple, quand un résultat contient un attribut de type *BenchString*, la somme n'étant pas pertinente. Elle retournera alors *MultiVal*.

Une instance de la classe *Benchmark* contient les structures des 4 types d'informations : une liste de profils, une liste de méthodes, une liste d'entrées et finalement les résultats.

La classe *Benchmark* est responsable de la gestion de ces informations et de leur lecture et écriture sur le disque.

D'autres fonctionnalités sont disponibles dans différents traits :

Analyze contient des outils pour analyser les résultats. Il permet, par exemple, d'obtenir une liste de profils ou seul un attribut défini est modifié. Cela peut être utilisé pour analyser l'évolution des résultats par rapport à cet attribut.

Visual est responsable de l'affichage des résultats dans différents types de graphiques

AutoRun Ce trait s'occupe de calculer les résultats pour toutes les combinaisons de profil, méthode et entrée. Pour ce faire, il est nécessaire de fournir au trait une fonction prenant comme paramètre un profil, une méthode, une entrée et un résultat sans valeur et retournant un résultat avec les valeurs assignées.

Cette fonction peut être renseignée de deux manières différentes : soit en assignant une fonction par méthode, soit en assignant une fonction générale. Lors de l'exécution d'un test, la fonction assignée à une méthode sera utilisée prioritairement.

Ce trait offre la possibilité de lancer un benchmark, de l'interrompre (volontairement ou non) et de le reprendre. Il permet également d'ajouter des profils, des méthodes ou des entrées et de recalculer facilement les informations manquantes.

En plus de ces traits, il existe une classe abstraite *BenchmarkQuick* qui, comme son nom l'indique, permet de paramétrer, lancer les tests et éventuellement de les relancer si un problème survient, de visualiser les résultats...

Dans l'exemple 2.5.1 suivant, nous paramétrisons un benchmark pour le problème *BACP* qui est expliqué dans la section 3.

Dans *setBenchmarkDomain* est configuré le benchmark, d'abord les structures des profils, des méthodes, des entrées et des résultats sont préparés. Ensuite sont ajoutés 3 méthodes, 99 entrées (correspondant au noms des fichiers contenant les instances) et 7 profils différents contenant les limites de temps après lesquelles les résolutions sont arrêtées, ce benchmark ayant pour but de montrer l'évolution du nombre d'instances pouvant être résolues pour différentes limites de temps.

La méthode *solve* (ici uniquement partiellement présente) est définie. Cette méthode sera appelée pour toute combinaison de profil, de méthode et d'entrée.

Exemple 2.5.1 Définition d'un benchmark

```
1 object BACXPX extends BenchmarkQuick {
2   final val SHAW = "Shaw"
3   final val CURRENT = "BinPackingFlow"
4   final val EXTENDED = "BinPackingFlowExtended"
5
6   override val benchTitle = Some("BACXPX")
7
8   def setBenchmarkDomain {
9
10    addEntryKey("file", "String")
11    addProfileKey("timeLimit", "Int")
12    addMethodKey("descr", "String")
13    addResultKey("optimalFound", "Int")
14    addResultKey("best", "Int")
15    addResultKey("backtracks", "Int")
16    addResultKey("time", "Double")
17
18    List(SHAW,CURRENT,EXTENDED).foreach(
19      d => addMethod(KeyedEntry(d)))
20    (1 to 99).foreach(
21      n => addEntry(KeyedEntry("inst" + n + ".txt")))
22    List(15, 30, 60, 120, 180, 240, 300).foreach(
23      t => addProfile(KeyedEntry(t)))
24  }
25
26  def solve(profile: Profile ,method: Method,
27    entry: EntryProfile ,result: Result) =
28  {
29    /*
30     * code to compute wath will be put in result
31     */
32    result.setValueForKey(
33      if (cp.objective.isOptimum) 1 else 0, "optimalFound")
34    result.setValueForKey(cp.objective.objs(0).best, "best")
35    result.setValueForKey(cp.bkts, "backtracks")
36    result.setValueForKey(cp.time, "time")
37
38    result
39  }
40
41  def main(args : Array[String]){
42    println(args)
43    args match{
44      case Array(x) if x startsWith "start" => start
45      case Array(x,f) if x startsWith "restart" => restart(f)
46      case Array(x,f) if x startsWith "visu"=> visualize(f)
47      case Array(x) => println("nothing to do")
48    }
49  }
50 }
```

La méthode *main*, quant à elle, permet, selon les arguments donnés, de lancer le benchmark, de prendre un fichier avec des résultats et de le compléter ou de visualiser les résultats d'un benchmark déjà réalisé.

Un exemple de visualisation pour l'exemple 2.5.1 est visible à la figure ?? . Sur cet exemple, on peut voir l'évolution de l'attribut *optimalFound* en fonction de l'attribut *timeLimit*. La zone de texte contient tous les profils obtenus avec un attribut variant. N'ayant qu'un attribut, la liste de profils égaux en omettant un attribut n'est composé que d'un seul profil. Si plusieurs profils étaient disponibles, le choix entre ces différents profils serait effectué grâce à la liste de sélection affichant *0*.

Les trois différentes fonctions sur le graphique représentent les différentes méthodes.

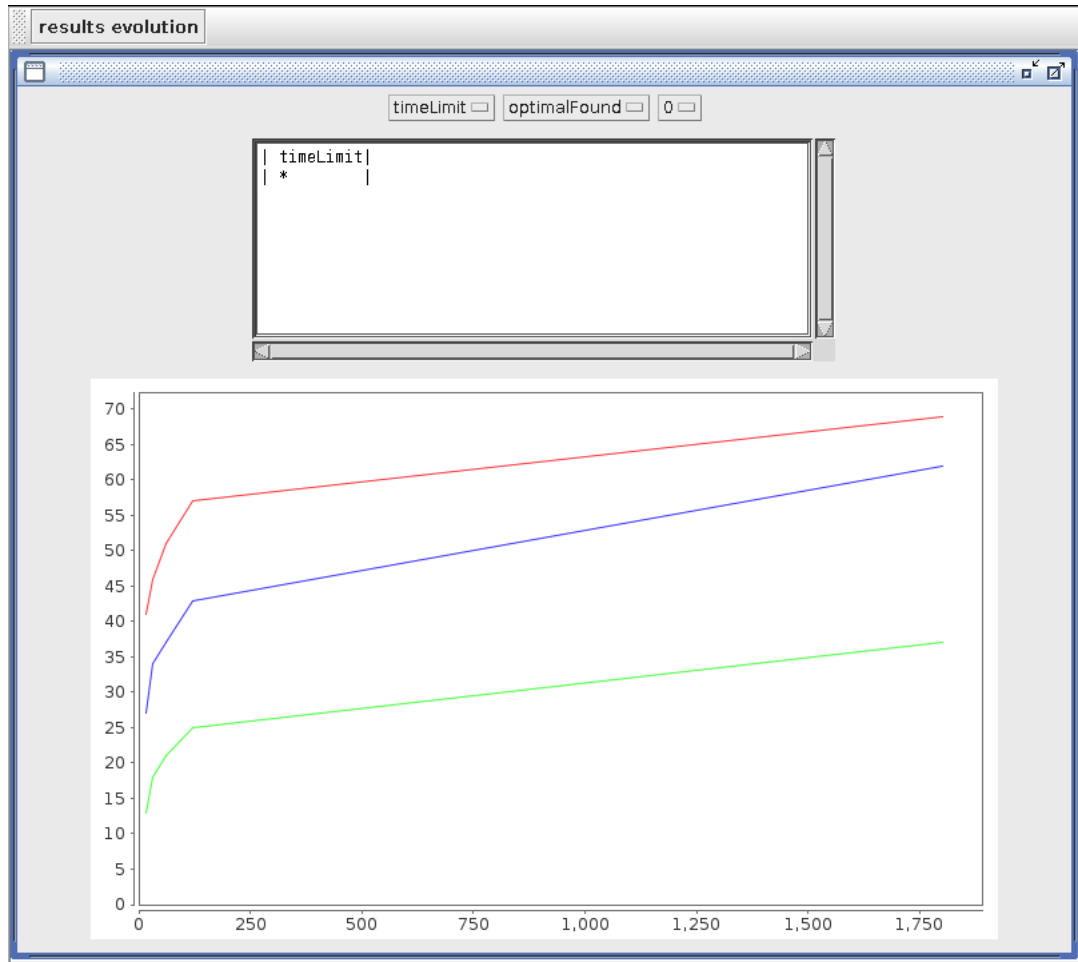


FIGURE 2.9: Evolution du nombre d'instances résolues en fonction de la limite de temps

2.5.4 Benchmark pour les différentes méthodes

Nous allons effectuer un benchmark sur les 720 instances et pour les 4 méthodes différentes. Chaque résolution est limitée à 5 minutes. Pour chaque instance et chaque méthode, nous essayerons de minimiser le nombre de nettoyages de cuves effectués. Nous ne prendrons pas le problème complet pour ce benchmark mais une abstraction ou les stabilités ne doivent être respectées.

Comme expliqué précédemment, en plus de trouver l’optimalité et de la prouver, trouver une solution faisable assez rapidement permet à un opérateur de négocier un cargo en connaissance de cause. C’est pourquoi, pour chaque instance, et pour les solveurs utilisant la programmation par contraintes, nous nous intéresserons au nombre d’instances pour lesquelles nous avons prouvé l’optimalité et le nombre d’instances pour lesquelles nous avons trouvé au moins une solution.

La table ?? montre pour différents sous-ensembles et pour chaque méthode le nombre d’éléments où l’optimalité est prouvée et où la faisabilité est prouvée (au moins une solution a été trouvée). On peut y voir que la programmation linéaire a de meilleurs résultats que la programmation par contrainte et que les différentes versions de la contrainte *BinPacking* n’ont pas de grande influence.

Instances	#INST	LP #OPT	CP1 #OPT	#FAIS	CP2 #OPT	#FAIS	CP3 #OPT	#FAIS
TAP	720	241	197	433	198	436	195	432
L20	240	104	81	164	81	165	80	163
L30	240	77	64	143	64	143	64	143
L40	240	60	52	126	53	128	51	126
C1	180	178	154	154	155	155	154	154
C2	180	39	33	109	33	109	33	108
C3	180	16	8	87	8	88	6	86
C4	180	8	2	83	2	84	2	84
F1	240	83	65	142	65	143	65	143
F2	240	80	67	140	67	142	66	140
F3	240	78	65	151	66	151	64	149
T24/S3	360	117	110	266	110	267	110	267
T24/S6	180	52	48	174	48	175	48	175
T38/S6	180	57	32	42	33	43	30	40
T38/S12	180	67	55	125	55	126	55	125

TABLE 2.1: Résultats généraux, pour les résultats issus de la programmation linéaire (LP), de la programmation par contraintes avec les différentes versions de *BinPacking* : celle issue de [8] (CP1), celle issue de [8] et de [7] et finalement celle de [8] et avec l’amélioration du filtrage appliquée à la contrainte de [7]

Pour comparer le temps nécessaire pour résoudre un problème, on prendra une moyenne contenant uniquement les temps de résolutions pour les instances où les quatre méthodes trouvent l’optimalité pour la solution. Le solveur utilisant la programmation linéaire met en moyenne 7s pour trouver la solution tandis que les solveurs utilisant la programmation par contraintes mettent respectivement 23s, 24ms et 36ms. Ces résultats mettent une fois de plus en évidence l’avantage du solveur LP.

Nous allons, à présent, comparer les différents solveurs CP. Grâce au tableau ??, nous nous rendons compte que, bien que les résultats soient fort semblables pour les trois différentes contraintes *BinPacking*, l’avantage va à la paire venant des techniques issues de [8] et de [7]. La contrainte développée dans le cadre de ce mémoire donne, quant à elle, des résultats légèrement inférieurs aux deux autres.

Nous pouvons expliquer la raison pour laquelle la contrainte modifiée dans le cadre de ce mémoire a donné des résultats un peu plus faibles que les autres. En effet, le fait d’améliorer le filtrage est coûteux en calculs. Le bateau ne comportant que peu de produits simultanément, les domaines des variables sont assez petits. Le filtrage sur la cardinalité n’apporte, dès

lors, pas un élagage important de l'arbre de recherche et l'amélioration des bornes de ces cardinalités n'ont pas assez d'effet que pour rentabiliser le coût nécessaire pour les calculer. Ceci est assez visible si on compare le nombre d'échecs lors de la résolution des instances pour les trois différentes stratégies de *BinPacking*. Pour ce faire, nous ne prendrons en compte que les instances où les trois solveurs ont trouvé l'optimalité. En effet, pour les autres instances, le nombre d'échecs est assez peu représentatif car les résolutions ne sont pas forcément arrêtées au même niveau. Nous avons respectivement 276531, 276481 et 276478 échecs. Nous voyons donc clairement que le nombre d'échecs est le plus faible pour le solveur avec la contrainte développée dans ce mémoire, mais la différence avec les autres solveurs, spécialement celui avec la contrainte issue de [7], est très faible.

2.6 Conclusion

Dans ce chapitre nous avons tout d'abord pris connaissance des problèmes liés au routage de chimiquier. Nous nous sommes ensuite concentré sur l'allocation des produits dans les cuves d'un bateau durant un voyage. Pour ce faire, nous avons repris le problème de [3].

Nous avons ensuite résolu ce problème en utilisant plusieurs techniques. Bien que les techniques utilisant la programmation par contraintes donnent des résultats inférieurs à la technique utilisant la programmation linéaire, elles ont d'autres avantages liés au fait que le modèle est plus "naturel". Ces modèles peuvent en effet être compris et modifiés plus facilement pour répondre à de nouvelles contraintes qui pourraient survenir.

Nous avons également créé des visuels permettant d'avoir une idée concrète des différentes solutions.

Chapitre 3

Contrainte *BinPacking*

3.1 Introduction

Lors de la modélisation du problème d'allocation des cuves dans un chimiquier à l'aide de la programmation par contraintes, une des contraintes principales était la contrainte globale *BinPacking*. Cette contrainte a été introduite dans la publication [8] et son filtrage a été adapté au problème d'allocation de cuves par l'article [7]. Lors du mémoire, cette dernière version a été modifiée afin d'avoir un meilleur élagage de l'arbre de recherche. Cependant, cela s'est fait au coût d'une complexité plus importante lors de la propagation.

Cette modification de la contrainte fait l'objet d'un article, écrit avec le professeur Pierre Schaus¹ et avec l'aide du professeur Jean-Charles Régin², article qui a été proposé pour publication lors de la conférence *CP 2013*³.

3.2 Contrainte BinPacking

La contrainte globale *BinPacking* $([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m])$ reproduit la situation de l'allocation de n objets pondérés indivisibles :

- X_i est une variable entière représentant la boîte où est placé l'objet i dont le poids strictement positif est w_i . Tous les objets doivent être placés, $Dom(X_i) \subseteq [1..m]$.
- L_j est une variable entière représentant la somme des poids des objets placés dans la boîte j .

La contrainte applique la relation suivante :

$$\forall j \in [1..m] : \sum_{i|X_i=j} w_i = L_j$$

Formulation classique Le moyen traditionnel de modéliser cette contrainte est de créer, pour chaque paire de boîte et d'objet une variable binaire $x_{i,j}$ qui vaut 1 si l'objet i est placé dans la boîte j c'est-à-dire :

$$\forall i \in [1..n], j \in [1..m] : x_{i,j} = 1 \iff X_i = j$$

1. UCLouvain, ICTEAM, Place Sainte-Barbe 2, 1348 Louvain-la-Neuve (Belgium), pierre.schaus@uclouvain.be

2. University of Nice-Sophia Antipolis, I3S UMR 6070, CNRS, (France) jcregin@gmail.com

3. <http://cp2013.a4cp.org/>

Les charges des boîtes sont maintenues par m produits scalaires, pour chaque boîte :

$$l_j = \sum_{i \in [1..n]} w_i x_{i,j}$$

Une contrainte redondante est finalement ajoutée, spécifiant que la somme des capacités des boîtes doit être égale à la somme des poids des objets :

$$\sum_{j \in [1..m]} l_j = \sum_{i \in [1..n]} L_j$$

Algorithmes existants L'algorithme de filtrage initial, proposé pour cette contrainte par Shaw dans [8], filtre essentiellement les domaines des variables X_i en utilisant un raisonnement proche de celle du problème du sac à dos pour détecter si forcer un élément dans une boîte j empêcherait d'atteindre la capacité de la boîte L_j . Cette procédure est très efficace mais retourne de faux positifs en disant que certains éléments sont acceptables pour une certaine boîte, alors qu'ils ne le sont pas. Un algorithme de détection d'échec a également été proposé dans [8]. Il calcule, pour chaque solution partielle, le nombre de boîtes nécessaires au minimum pour compléter cette solution partielle. Cette vérification de la consistance a été étendue dans [2]. Cambazard et O'Sullivan [1] ont proposé de filtrer les domaines en utilisant une formulation de programmation linéaire basée sur les flots.

3.3 Réflexion sur les cardinalités

Dans les problèmes classiques utilisant la contrainte *BinPacking*, la capacité maximale de chaque boîte $\max(L_j)$ est, en général, contrainte alors que la capacité minimale $\min(L_j)$ est fixée à 0. C'est la raison pour laquelle les algorithmes de filtrage existants se concentrent sur la borne maximale $\max(L_j)$ et ne se concentrent pas spécialement sur les bornes minimales des capacités $\min(L_j)$.

Cependant, pour le problème de l'allocation des cuves d'un chimiquier, les bornes minimales $\min(L_j)$ sont les bornes importantes.

Pour cette raison, l'article [7] a introduit un filtre basé sur les cardinalités. Son principe est de compter le nombre d'objets nécessaires pour chaque boîte. Cette contrainte peut être vue comme la généralisation de la contrainte *BinPacking* suivante :

$$\text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m], [C_1, \dots, C_m])$$

où les variables $[C_1, \dots, C_m]$ contiennent le nombre d'objets assigné à chaque boîte. En d'autres termes, ils respectent la relation : $\forall j \in [1..m] : C_j = |\{i | X_i = j\}|$. Cette formulation est particulièrement adaptée pour les problèmes comportant ces particularités :

- La borne inférieure de la variable de capacité des boîtes est également initialement contrainte par $\min(L_j) > 0$.
- Les objets à placer dans les boîtes ont approximativement le même poids (la contrainte est dominée par un problème d'assignation).
- il y a des contraintes portant sur le nombre d'objets dans chaque boîte.

L'idée de [8] est d'introduire une contrainte globale redondante [?] :

$$\begin{aligned} \text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m], [C_1, \dots, C_m]) \equiv \\ \text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m]) \wedge \\ \text{GCC}([X_1, \dots, X_n], [C_1, \dots, C_m]) \end{aligned}$$

Un algorithme spécialisé sera utilisé pour ajuster les bornes supérieures et inférieures des variables C_j à chaque fois que les bornes des L_j changent et/ou que les domaines des X_i changent.

Plus les bornes calculées pour les variables de cardinalité sont serrées et meilleur sera le filtrage apporté par la contrainte *GCC*.

Avant de voir l'algorithme utilisé dans [7], quelques définitions sont nécessaires :

Définition 3.3.1 *L'ensemble des objets déjà placés dans la boîte j est noté $pack_j = \{i | Dom(X_i) = \{j\}\}$ et l'ensemble des objets candidats disponibles pour aller dans la boîte j est noté $cand_j = \{i | j \in Dom(X_i) \wedge |Dom(X_i)| > 1\}$. La somme des poids des objet présents dans le set S est $sum(S) = \sum_{i \in S} w_i$.*

Dans l'article [7], la borne minimale sur le nombre d'objets qui peuvent être placés dans une boîte est obtenue en trouvant la taille de l'ensemble de cardinalité minimale $A_j \subseteq cand_j$ tel que $sum(A_j) \geq min(L_j) - sum(pack_j)$. De là, on tire $min(C_j) \geq |pack_j| + |A_j|$. On peut, dès lors, filtrer la borne minimale de la cardinalité C_j de la manière suivante :

$$min(C_j) \leftarrow \max(min(C_j), |pack_j| + |A_j|).$$

Cet ensemble A_j est obtenu dans l'article [7] par un algorithme glouton qui scanne les objets présents dans $cand_j$ triés par poids croissant jusqu'à ce que le poids accumulé des objets de $min(L_j) - sum(pack_j)$ soit atteint. Ceci est représenté dans l'algorithme 1.

Algorithme 1 : Computes a lower bound on the cardinality of bin j with the method from [7]

Data : j a bin index
Result : $binMinCard$ a lower bound on the min cardinality for the bin j

```

1  $binLoad \leftarrow sum(pack_j)$  ;
2  $binMinCard \leftarrow |pack_j|$  ;
3 foreach  $i \in cand_j$  in decreasing weight order do
4   if  $binLoad \geq min(L_j)$  then
5      $\quad$  break ;
6    $binLoad \leftarrow binLoad + w_i$  ;
7    $binMinCard \leftarrow binMinCard + 1$  ;
8 if  $binLoad < min(L_j)$  then
9    $\quad$  The constraint is unfeasible ;
```

Comme les objets sont triés initialement, cet algorithme s'exécute en un temps linéaire et la complexité est donc $O(n)$ où n est le nombre d'objets.

Un raisonnement similaire est utilisé pour calculer la borne maximale de la variable de cardinalité $max(C_j)$.

Exemple 3.3.1 Calcul de la cardinalité minimale pour une boîte

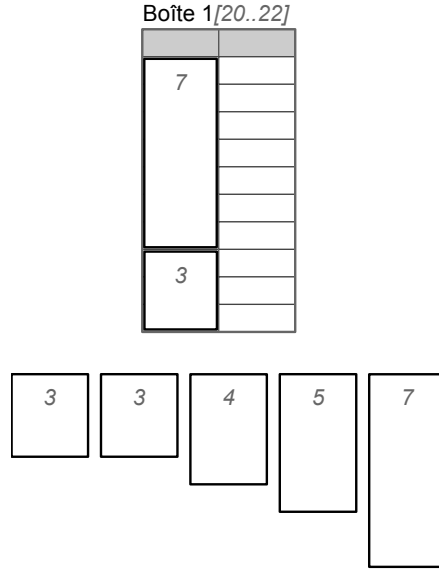
Cinq objets pondérés 3, 3, 4, 5, 7 peuvent être placés dans la boîte 1 qui a une capacité possible $L_1 \in [20..22]$. Deux autres objets sont déjà placés dans cette boîte avec les poids 3 et 7 ($|pack_1| = 2$ and $l_1 = 10$). L'algorithme trouvera $|A_1| = 2$ obtenu avec les objets de poids 5, 7. La valeur minimale du domaine de la variable de cardinalité C_1 est donc mise à 4. Cette situation est représentée à la figure 3.1

3.4 Amélioration du filtrage de la contrainte

3.4.1 Filtrage sur les variables de cardinalité

Le calcul de la borne inférieure (supérieure) de la cardinalité C_j introduite dans [7] ne considère que les objets candidats $cand_j$ ainsi que la capacité minimale (maximale) à attein-

FIGURE 3.1: Instance de la contrainte *BinPacking* avec une boîte et 7 produits ; les cases claires de la boîte sont obligatoires tandis que les cases gris claires sont optionnelles



dre, à savoir $\min(L_j)$ ($\max(L_j)$). Des bornes plus fortes peuvent être trouvées si on considère également les variables de cardinalité des autres boîtes. En effet, un objet qui est utilisé pour atteindre la cardinalité minimale pour une boîte peut ne pas être disponible pour une autre boîte. Cela sera illustré dans les exemples suivants.

Exemple 3.4.1 Intuition sur le calcul de la cardinalité minimale avec la méthode modifiée

L'instance utilisée pour cet exemple est représentée à la figure 3.2. La boîte 1 peut accepter des objets ayant les poids 3, 3, 3 avec une capacité minimale de 6 et donc une cardinalité de 2 objets. La boîte 2, quant à elle, a une capacité minimale de 5 et peut accepter les mêmes objets que la boîte 1 ainsi que deux objets de poids 1. La boîte 2 ne peut clairement pas utiliser plus d'un objet avec un poids 3 pour calculer la borne inférieure de sa cardinalité. Cela empêcherait effectivement la boîte 1 d'atteindre sa cardinalité minimale de 2. Ainsi, la cardinalité minimale de la boîte 2 devrait être 3 et non 2 comme cela serait calculé avec la borne minimale de [7].

L'algorithme 2 calcule une cardinalité minimale pour la boîte j plus forte que celle calculée par [7] en prenant également en compte les variables de cardinalité des autres boîtes $\min(C_k) \forall k \neq j$.

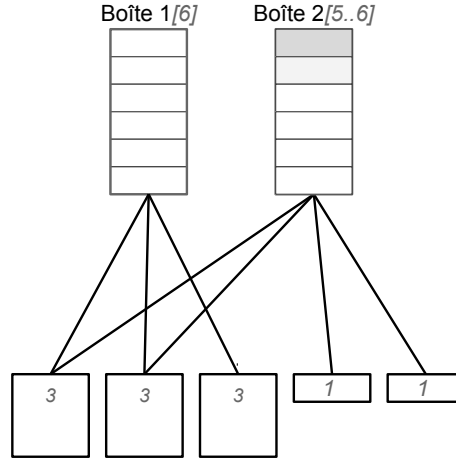
Il empêche de réutiliser un objet s'il est requis par une autre boîte pour atteindre sa cardinalité minimale. Ceci est réalisé en gardant, pour chaque autre boîte k , le nombre d'objets que cette boîte est capable de donner sans l'empêcher de compléter l'exigence de sa propre cardinalité minimale $\min(C_k)$.

Si une boîte k doit contenir au moins $\min(C_k)$ objets pour atteindre sa capacité minimale et qu'elle en contient déjà $|pack_j|$, il est évident que cette boîte ne peut pas donner plus de $|cand_k| - (\min(C_k) - |pack_k|)$ objets à la boîte j . Cette information est maintenue dans la variable *availableForOtherBins_k*, initialisée à la ligne 5.

Exemple 3.4.2

Pour continuer l'exemple 3.4.1, la boîte 1 va avoir $availableForOtherBins_1 = 3 - (2 - 0) = 1$ car cette boîte peut donner, au maximum, un de ces objets à une autre boîte.

FIGURE 3.2: Instance avec deux boîtes et cinq produits ; les cases claires de la boîte sont obligatoires alors que les cases gris claires sont optionnelles. Les arcs représentent la compatibilité entre les boîtes et les objets.



Comme les objets sont triés par poids décroissant à la ligne 7, les autres boîtes acceptent de donner leurs objets candidats les plus lourds en premier. C'est une situation optimiste du point de vue de la boîte j . Cela justifie le fait que l'algorithme calcule une borne minimale valide pour la variable de cardinalité C_j . Chaque fois qu'un objet est utilisé par la boîte j , les autres boîtes (où cet objet est un candidat) diminuent la quantité *candidatesAvailableForBin_k* car elles "consommement" leur capacité de partager un objet. Un objet n'est pas disponible, si au moins une autre boîte k n'a plus d'objet à partager, c'est-à-dire qu'elle ne pourrait pas se passer de tous les objets faisant partie de ses candidats *cand_k* et étant utilisés pour trouver la cardinalité minimale de la boîte j . Ceci est détecté à la ligne 13 : *available* est mis à *false*, ce qui signifie que cet objet ne sera pas utilisé pour le calcul de la borne minimale de la cardinalité de la boîte j nécessaire pour atteindre la capacité minimale requise *min(L_j)*.

Si par contre l'objet peut être utilisé (*available=true*), alors, les autres boîtes qui ont accepté de donner un objet, ont un objet en moins disponible. Les valeurs de *availableForOtherBins_k* sont décrémentées à la ligne 22. Finalement, à la ligne 28, l'algorithme peut détecter des situations où la contrainte n'est pas réalisable quand on ne peut atteindre une capacité minimale pour la boîte.

Algorithme 2 : Calcule la borne inférieure sur la cardinalité de la boîte j

```

Data :  $j$  a bin index
Result :  $binMinCard$  a lower bound on the min cardinality for the bin  $j$ 
1  $binLoad \leftarrow \text{sum}(\text{pack}_j)$  ;
2  $binMinCard \leftarrow |\text{pack}_j|$  ;
3  $\text{othersBins} \leftarrow \{1, \dots, m\} \setminus j$  ;
4 foreach  $k \in \text{othersBins}$  do
5    $\text{availableForOtherBins}_k \leftarrow |\text{cand}_k| - (\min(C_k) - |\text{pack}_k|)$ ;
6 foreach  $i \in \text{cand}_j$  in decreasing weight order do
7   if  $binLoad \geq \min(L_j)$  then
8      $\text{break}$  ;
9    $\text{available} \leftarrow \text{true}$ ;
10  for  $k \in \text{othersBins}$  do
11    if  $k \in \text{Dom}(X_i) \wedge \text{availableForOtherBins}_k = 0$  then
12       $\text{available} \leftarrow \text{false}$  ;
13  if  $\text{available}$  then
14     $binLoad \leftarrow binLoad + w_i$  ;
15     $binMinCard \leftarrow binMinCard + 1$  ;
16    for  $k \in \text{othersBins}$  do
17      if  $k \in \text{Dom}(X_i)$  then
18         $\text{availableForOtherBins}_k \leftarrow \text{availableForOtherBins}_k - 1$  ;
19 if  $binLoad < \min(L_j)$  then
20    $\text{The constraint is unfeasible}$  ;

```

Cardinalité maximale L'algorithme pour calculer la cardinalité maximale d'une boîte est similaire à l'algorithme 2, à l'exception de quelques changements :

1. La variable $binMinCard$ doit être renommée $binMaxCard$
2. Les objets sont considérés dans un ordre croissant de poids à la ligne 7
3. Le critère d'arrêt à la ligne 9 devient $binLoad + w_i > \max(L_j)$
4. Il n'y a pas de test de faisabilité aux lignes 27-29

Complexité En supposant que les objets sont initialement triés dans un ordre de poids décroissant, cet algorithme a une complexité de $\mathcal{O}(n \cdot m)$ avec n le nombre d'objets à placer et m le nombre de boîtes. Ajuster la cardinalité de toutes les boîtes prend donc $\mathcal{O}(n \cdot m^2)$. Cet algorithme n'est pas garanti d'être idempotent. En effet, la boîte j peut considérer un objet i comme disponible. Toutefois, après avoir ajusté la cardinalité minimale d'une autre boîte k , cet objet peut devenir indisponible si la boîte j est à nouveau considérée. Nous devons, dès lors, appeler la méthode de propagation non seulement lorsque les bornes des L_j changent et/ou que les domaines des X_i changent, mais aussi lorsque les bornes de des C_j changent.

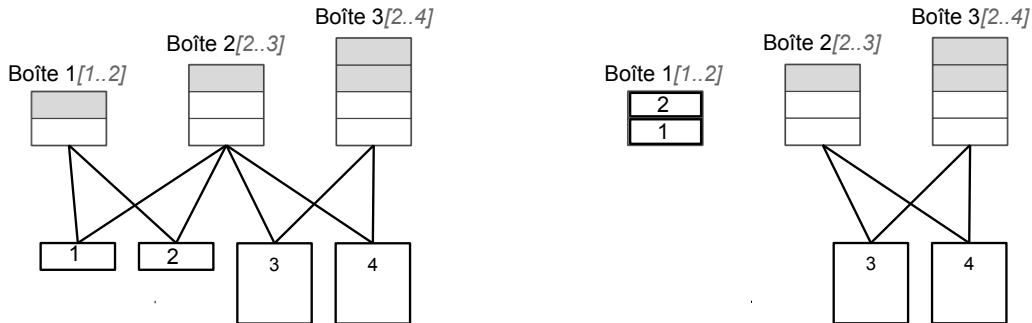


FIGURE 3.3: (a) Instance *BinPacking* avec 3 boîtes et 4 objets. Les arcs représentent les boîtes possibles pour chaque objet. (b) Domaines résultant du filtrage avec les bornes calculées à partir de l'algorithme décrit dans la section 3.4.

Exemple 3.4.3 Différences de filtrage entre les deux algorithmes

L'instance considérée est représentée à la figure 3.3(a)

$$\begin{aligned} \text{BinPacking}([X_1, \dots, X_4], [w_1, \dots, w_4], [L_1, \dots, L_3]) \\ X_1 \in \{1, 2\}, X_2 \in \{1, 2\}, X_3 \in \{2, 3\}, X_4 \in \{2, 3\}, \\ w_1 = 1, w_2 = 1, w_3 = 3, w_4 = 3 \\ L_1 \in \{1, 2\}, L_2 \in \{2, 3\}, L_3 \in \{2, 4\} \end{aligned}$$

On considère d'abord le calcul de la cardinalité de la boîte 2. Cette boîte doit avoir au moins un objet pour atteindre sa capacité minimale. On considère maintenant la cardinalité maximale de cette boîte. Les objets 1 et 2 peuvent tous les deux être placés dans la boîte 2. Cependant, le faire empêcherait la boîte 1 d'atteindre sa charge minimale requise de 1. Dès lors, seul un de ces objets peut être utilisé lors du calcul de la borne maximale de la boîte 2. En considérant que l'objet 1 est utilisé, le prochain objet à considérer est l'objet 3 ayant un poids de 3. Toutefois, ajouter cet objet avec l'objet 1 déjà présent excéderait la charge maximale ($4 > 3$) (critère d'arrêt pour le calcul de la cardinalité maximale). C'est pourquoi la cardinalité maximale finale pour la boîte 2 est 1. Le raisonnement sur la cardinalité trouve également que la boîte 1 doit avoir entre un et deux objets que la boîte 3 doit en avoir exactement un. Basé sur ces cardinalités, la contrainte globale de cardinalité (*GCC*) est capable de déduire que les objets 1 et 2 doivent être placés dans la boîte 1. Ce filtrage est représenté sur la figure 3.3 (b).

L'algorithme de [7] déduit que la boîte 2 doit avoir entre un et deux objets (alors que le nouveau filtrage trouve qu'il en faut exactement deux). La borne supérieure de deux objets est utilisée avec les deux objets les plus légers, les objets 1 et 2. Comme pour le nouvel algorithme, il déduit que la boîte 1 doit avoir entre un et deux objets et que la boîte 3 doit avoir exactement un objet. Malheureusement, la contrainte *GCC* n'est pas capable de retirer de boîtes du domaine des objets en se basant sur les bornes des cardinalités. Nous restons donc avec la situation décrite à la figure 3.3 (a). En somme, cet algorithme est moins puissant que le nouveau.

3.4.2 Filtrage sur les variables de capacité

Un filtrage sur les variables de capacité des boîtes est introduit en prenant en compte l'information apportée par la variable de cardinalité. Ce filtrage n'était pas proposé dans l'article [7]. L'algorithme 3 est similaire à l'algorithme 2, excepté que son but est de trouver la borne inférieure de la capacité de la boîte. Pour ce faire, l'algorithme choisit d'abord les objets de poids faible jusqu'à atteindre la cardinalité minimale pour la boîte en question. Un raisonnement similaire peut, à nouveau, être utilisé pour calculer la borne supérieure de la capacité de la boîte.

Algorithme 3 : Calcule la borne inférieure de la capacité de la boîte j

Data : j a bin index
Result : $binMinLoad$ a lower bound on the load of bin j

```

1  $binCard \leftarrow \lfloor pack_j \rfloor$  ;
2  $binMinLoad \leftarrow sum(pack_j)$  ;
3  $othersBins \leftarrow \{1, \dots, m\} \setminus j$  ;
4 foreach  $k \in otherBins$  do
5    $availableForOtherBins_k \leftarrow \lfloor cand_k \rfloor - (\min(C_k) - \lfloor pack_k \rfloor)$ ;
6 foreach  $i \in cand_j$  in increasing weight order do
7   if  $binCard \geq \min(C_j)$  then
8      $\lfloor$  break ;
9   available  $\leftarrow$  true;
10  for  $k \in othersBins$  do
11    if  $k \in Dom(X_i) \wedge availableForOtherBins_k = 0$  then
12       $\lfloor$  available  $\leftarrow$  false ;
13  if available then
14     $binMinLoad \leftarrow binLoad + w_i$  ;
15     $binCard \leftarrow binCard + 1$  ;
16    for  $k \in othersBins$  do
17      if  $k \in Dom(X_i)$  then
18         $\lfloor$   $availableForOtherBins_k \leftarrow availableForOtherBins_k - 1$  ;
19 if  $binCard < \min(C_j)$  then
20    $\lfloor$  The constraint is unfeasible ;

```

limite (s)	A	B	C
15	13	27	41
30	18	34	46
60	21	37	51
120	25	43	57
1800	37	62	69

TABLE 3.1: Nombre d'instances pour lesquelles il a été possible de prouver l'optimalité dans la limite de temps

3.5 Expérimentations

Les expérimentations sont faites sur le problème *Balanced Academic Curriculum Problem* (BACP) qui est récurrent dans les universités. Le but est de planifier les cours qu'un étudiant doit suivre de manière à respecter les prérequis entre les cours et d'équilibrer le plus possible la charge de travail entre les différentes périodes. Chaque période est également limitée à un nombre minimum et maximum de cours. La plus grande des 3 instances disponibles sur CSPLIB⁴ a 12 périodes, 66 cours ayant un poids compris entre 1 et 5 crédits et 65 relations de prérequis. Elle a été modifiée dans [6] pour générer 100 nouvelles instances⁵ en donnant aléatoirement un coût compris entre 1 et 5 crédits à chaque cours et en gardant, de manière aléatoire également, 50 des 65 relations de prérequis. Chaque période doit se composer de 5 à 7 cours. Comme montré dans [4], une meilleure propriété d'équilibrage est obtenue en

4. <http://www.csplib.org>

5. Disponibles sur <http://becool.info.ucl.ac.be/resources/bacp>

minimisant la variance à la place de la charge maximale. Pour chaque instance, on teste 3 configurations de filtrage différentes pour la contrainte *BinPacking* :

- A : la contrainte issue de [8] + une contrainte *GCC*
- B : A + le filtrage sur la cardinalité issu de [7]
- C : A + le filtrage sur la cardinalité qui est expliquée dans la section 3.4

La table 3.1 donne le nombre d’instances résolues pour différentes limites de temps. La table 3.2, quant à elle, illustre les statistiques plus détaillées (temps, meilleure borne, nombre d’échecs) pour certaines instances avec une limite de temps de 30 minutes. On peut voir que le nouveau filtrage permet de résoudre davantage d’instances et, parfois, de réduire le nombre d’échecs par plusieurs ordres de grandeurs.

instance	temps (ms)			meilleure borne			nombre d’échecs		
	A	B	C	A	B	C	A	B	C
inst2.txt	timeout	timeout	679	3243	3247	3237	835459	1064862	829
inst14.txt	timeout	45625	6925	3107	3105	3105	1043251	228294	8530
inst22.txt	timeout	13971	281	3045	3041	3041	811852	48482	353
inst30.txt	timeout	118964	192	3416	3402	3402	795913	707487	129
inst36.txt	timeout	timeout	337	2685	2685	2671	847641	915849	364
inst47.txt	timeout	timeout	112	3309	3309	3303	2561038	3812512	269
inst65.txt	timeout	timeout	222	3416	3414	3402	921694	1091396	168
inst70.txt	timeout	timeout	101060	3043	3043	3041	1917729	1516627	125270
inst87.txt	16275	15089	251	3643	3643	3643	109173	65493	207
inst98.txt	timeout	timeout	48	2987	2987	2979	7023383	8261509	261

TABLE 3.2: Statistiques détaillées de certaines instances obtenues sur certaines instances clés

3.6 Combiner des boîtes pour des bornes encore plus fortes

L’algorithme décrit dans la section 3.4 pourrait encore utiliser plus d’informations disponibles lors du calcul de la borne inférieure de la variable de cardinalité. L’exemple 3.6.1 décrit une situation où ces bornes pourraient être améliorées.

Exemple 3.6.1

L’instance de la contrainte *BinPacking* utilisée pour cet exemple est représentée à la figure 3.4. Elle est composée de 3 boîtes ayant des capacités de 2, de deux objets de poids 2 pouvant aller dans toutes les boîtes, ainsi que de deux objets de poids 1 ne pouvant aller que dans la troisième boîte.

L’algorithme, tel qu’il est décrit dans la section 3.4, trouverait la borne inférieure de la variable de cardinalité de la boîte 3 : $\min(C_3) = 1$. En effet, les deux premières boîtes peuvent partager toutes les deux un objet. Elles vont donc toutes les deux partager un objet de poids 2 qui sera accessible pour la boîte 3 lors du calcul de la cardinalité minimale. Cependant, il est clair qu’il faut les deux objets de poids 1 pour remplir la boîte 3 étant donné que les 2 premières boîtes nécessitent en réalité les deux objets de poids 2.

Dans cet exemple, la cardinalité exacte de la troisième boîte pourrait être trouvée en combinant les deux premières boîtes. En effet, ensemble, les deux premières boîtes possèdent 2 candidats mais nécessitent également 2 objets ; elles ne peuvent, dès lors, pas en partager.

L’exemple 3.6.1 peut être généralisé de la manière suivante : soit B un ensemble de boîtes, l’ensemble des objets candidats pour ces boîtes est : $cand_B = \bigcup_{b \in B} cand_b$, l’ensemble des objets déjà placés dans ces boîtes est : $pack_B = \bigcup_{b \in B} pack_b$ et la cardinalité minimale des boîtes présentes dans B , c’est-à-dire le nombre minimal d’objets nécessaires pour remplir toutes les boîtes de l’ensemble B est : $\min(C_B) = \sum_{b \in B} \min(C_b)$. Le nombre d’objets que l’ensemble des boîtes B peut partager est, dès lors : $cand_B - (\min(C_B) - pack_B)$.

L’algorithme de calcul des cardinalités des boîtes pourrait être adapté pour filtrer non seulement les objets nécessaires aux autres boîtes mais également à tous les groupements

possibles des autres boîtes. Cela nous permettrait de trouver de meilleures bornes pour les variables de cardinalité des boîtes. Cependant le nombre de combinaisons possibles évolue de façon factorielle par rapport au nombre de boîtes. Nous aurions donc une complexité de $\mathcal{O}(n \cdot m!)$, avec n le nombre d'objets et m le nombre de boîtes, pour calculer une borne de la variable de cardinalité d'une boîte. Il faudrait également garder en mémoire ou recréer à chaque itération les ensembles $pack_B$ et $cand_B$ pour chaque combinaison de boîtes. Le coût en calcul effectué de l'amélioration des bornes est, dès lors, très élevé et n'est pas acceptable; en tout cas, pas dans les problèmes qui ont été envisagés.

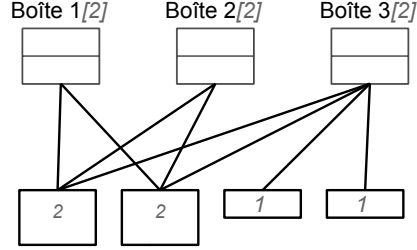


FIGURE 3.4: Instance de la contrainte *BinPacking* avec 3 boîtes et 4 produits

3.7 Conclusion

Nous avons introduit, lors du calcul, des cardinalités plus fortes sur la contrainte *BinPacking* en intégrant les exigences sur les cardinalités pour les autres boîtes. Ces bornes plus fortes ont un impact direct sur les variables de placement par l'intermédiaire de la contrainte *GCC*. Le filtrage amélioré a été expérimenté sur les problème BACP permettant de résoudre plus d'instances et de réduire considérablement le nombre d'échecs pour certaines instances.

Conclusion

La première phase de ce mémoire a été consacrée à l'apprentissage du langage Scala et à l'utilisation de la programmation fonctionnelle, entre autres grâce au cours *Fonctionnal Programmin Principles in Scala*⁶ donné par Martin Odersky sur coursera⁷. Suite à cela il a fallu se familiariser avec Oscar, librairie contenant des outils de résolution de problèmes de recherche opérationnelle. L'apprentissage de ces deux outils a été très enrichissant.

Par la suite, nous avons modélisé le problème décrit dans l'article [3] d'abord, comme il l'avait été fait dans l'article, en utilisant la programmation linéaire, ensuite en utilisant la programmation par contraintes. Nous sommes arrivés à certains résultats pour le problème modélisé par la programmation par contraintes mais ils ne sont pas encore à la hauteur de ceux correspondant à la programmation linéaire. De plus, nous utilisons une abstraction du problème et non pas le problème dans son entièreté car la programmation par contraintes n'est pas réellement adaptée aux contraintes de stabilité qui étaient présentes dans le problème de base.

L'étape suivante a été l'évolution de la contrainte décrite dans [7]. Bien que cette contrainte ne se soit pas montrée intéressante pour le problème d'allocation des cuves sur lequel nous travaillions, elle s'est avérée très efficace sur le problème consistant à trouver un curriculum académique équilibré. Ces bons résultats nous ont poussés à écrire un article sur l'évolution de cette contrainte qui a été soumis, pour publication, dans le cadre de la conférence *CP 2013*⁸. Les premiers commentaires que nous avons reçus sont assez encourageantes. Cet article se trouve en annexe du mémoire.

Pour finir, certains outils ont été réalisés durant ce mémoire. En outre, certains outils de visualisation ont été créés ou portés de Java à Scala et font déjà partie d'Oscar. Un outil de réalisation de *benchmarks* a également été développé.

La prochaine étape dans la modélisation du modèle par contraintes de l'allocation de produits aux cuves d'un chimiquier pourrait être une gestion des contraintes de stabilité dans le modèle même. En effet, la solution que nous avons adopté, qui consiste à vérifier la stabilité après l'allocation faite, ne donne pas des résultats concluants.

Ce travail m'a confirmé mon intérêt pour la recherche opérationnelle et pour la programmation par contraintes plus précisément. Les sources écrites pour ce mémoire, composées en grande partie de nouvelles productions mais aussi de quelques modifications de code existant et de traductions de classe Java en Scala représentent un total de 4178 lignes de code ou de commentaires pour plus de 50 classes.

6. <https://www.coursera.org/course/progfun>

7. <https://www.coursera.org/>

8. <http://cp2013.a4cp.org/>

Bibliographie

- [1] Hadrien Cambazard and Barry O'Sullivan. Propagating the bin packing constraint using linear programming. In *Principles and Practice of Constraint Programming-CP 2010*, pages 129–136. Springer, 2010.
- [2] Julien Dupuis, Pierre Schaus, and Yves Deville. Consistency check for the bin packing constraint revisited. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 117–122. Springer, 2010.
- [3] Kjetil Fagerholt. Ship routing and scheduling : Theory and practice. 2009.
- [4] Lars Magnus Hvattum, Kjetil Fagerholt, and Vinícius Amaral Armentano. Tank allocation problems in maritime bulk shipping. *Computers & OR*, 36(11) :3051–3060, 2009.
- [5] Jean-Noël Monette, Pierre Schaus, Stéphane Zampelli, Yves Deville, and Pierre Dupont. A CP approach to the balanced academic curriculum problem. In *Seventh International Workshop on Symmetry and Constraint Satisfaction Problems*, volume 7, 2007.
- [6] Oscar Team. Oscar : Scala in or, 2012. Available from <https://bitbucket.org/oscarlib/oscar>.
- [7] Jean-Charles Régin. Generalized arc consistency for global cardinality constraint. In *Proceedings of the thirteenth national conference on Artificial intelligence-Volume 1*, pages 209–215. AAAI Press, 1996.
- [8] Pierre Schaus et al. *Solving balancing and bin-packing problems with constraint programming*. PhD thesis, PhD thesis, Universit catholique de Louvain Louvain-la-Neuve, 2009.
- [9] Pierre Schaus, Jean-Charles Régin, Rowan Van Schaeren, Wout Dullaert, and Birger Raa. Cardinality reasoning for bin-packing constraint : application to a tank allocation problem. In *Principles and Practice of Constraint Programming*, pages 815–822. Springer, 2012.
- [10] Paul Shaw. A constraint for bin packing. In *Principles and Practice of Constraint Programming-CP 2004*, pages 648–662. Springer, 2004.

Annexe

Sources produites dans le cadre du mémoire

Les sources produites dans le cadre de ce mémoire sont disponibles sur le site <http://bitbucket.org/fpelsser/memoire>.

Etant donné que certaines sources sont dispersées dans le code d'Oscar, une copie de ces sources est disponible. Toutefois, elles ne peuvent être compilées.

Une version déjà compilée est fournie sous la forme d'une archive jar d'Oscar contenant toutes les sources.

Le dossier *data* contient les instances du problème *BACP* et *TAP*. Le dossier *lib* contient les bibliothèques nécessaires pour les exécutions nécessitant la programmation linéaire et le dossier *sources* contient les sources utilisées pour ce mémoire.

Les sources sont divisées en plusieurs dossiers :

contraintes Certaines contraintes créées dans le cadre de ce mémoire

visuel Certains visuels créés, mis à jour ou simplement traduits de Java vers Scala

exemples TAP Tous les éléments nécessaires pour résoudre le problème d'allocation des cuves.

BinPacking Des classes de tests et de benchmarks pour la contrainte *BinPacking*. Lors du passage à la dernière version de Scala, les acteurs Scala que certaines portions de code utilisaient ont été retirés. Cela rend une partie de ce code inutilisable car nous avons commenté certaines portions de code empêchant la compilation. Les benchmarks posant problèmes n'ont finalement pas été utilisés car le problème *BACP* donnait de meilleurs résultats. Ces classes n'ont pas été mises à jour.

BACPXP La classe permettant de faire des benchmarks de différentes techniques d'implémentation de la contrainte *BinPacking* pour le problème *BACP*.

benchmarks Le paquet benchmarks créé pour faciliter la création de benchmarks

Nous allons décrire la manière d'exécuter certains tests. Ces exemples sont écrits pour être lancés depuis un terminal sous Mac Os depuis la racine du répertoire pouvant être téléchargé sur le site indiqué précédemment ; c'est-à-dire le répertoire contenant *oscar.jar*. Ces exemples doivent être adaptés pour être exécutés sous d'autres systèmes d'exploitation. Les versions des bibliothèques sont disponibles pour les versions 64 bits de Linux, Mac Os et Windows ainsi que pour la version 32 bits de Windows. Il est nécessaire de bien être situé dans le bon dossier car les chemins des fichiers ne seraient, le cas échéant, pas trouvés. TM

- Lancer *TAPRunner* et résoudre des instances du problème d'allocation de cuves. Pour ce faire, il faut exécuter la commande :

```
1 java -Djava.library.path=lib/macos64/ -cp oscar.jar oscar.examples.TAP.  
   TAPRunner
```

Un invite de commande s'ouvrira alors. Trois commandes sont disponibles : *list*, *run*, *exit*. *list* permet de lister toutes les instances disponibles, chaque instance est numérotée. Il devrait y avoir des instances numérotées de 0 à 719. La commande *run* permet de lancer la résolution d'une instance. Cette commande prend un argument qui est le numéro de l'instance à résoudre. Plusieurs solveurs seront alors proposés, l'indice du solveur désiré doit être entré. Pour comparer plusieurs solveurs, il faut entrer leurs indices séparés par un espace. A la fin de la résolution, la possibilité de visualiser les résultats est offerte. La commande *exit*, quant à elle, quitte le programme.

- Les benchmarks correspondant aux problèmes *TAP* et *BACP* peuvent être lancés avec les commandes

```
1 java -Djava.library.path=lib/macos64/ -cp oscar.jar oscar.examples.TAP.  
   TAPBenchmark start
```

et

```
1 java -Djava.library.path=lib/macos64/ -cp oscar.jar oscar.examples.cp.BACXP  
   start
```

Cette commande va lancer le benchmark et créer, dès le premier résultat obtenu, un fichier dans le dossier courant contenant le benchmark partiel.

L'argument *start* peut être remplacé par *restart* accompagné d'un chemin vers un fichier contenant un benchmark partiel afin de le reprendre là où il avait été interrompu.

- Un test de la visualisation de l'évolution des résultats d'un benchmark fictif peut être obtenu avec la commande

```
1 java -Djava.library.path=lib/macos64/ -cp oscar.jar oscar.benchmarks.  
   BenchmarkVisualTest
```

Revisiting the cardinality reasoning for BinPacking constraint

François Pelsser¹, Pierre Schaus¹, Jean-Charles Régin²

¹ UCLouvain, ICTEAM,
Place Sainte-Barbe 2,
1348 Louvain-la-Neuve (Belgium),
`pierre.schaus@uclouvain.be`

² University of Nice-Sophia Antipolis,
I3S UMR 6070, CNRS, (France)
`jcregin@gmail.com`

Abstract. In a previous work, we introduced a filtering for the Bin-Packing constraint based on a cardinality reasoning for each bin combined with a global cardinality constraint. We improve this filtering with an algorithm providing tighter bounds on the cardinality variables. We experiment it on the Balanced Academic Curriculum Problems demonstrating the benefits of the cardinality reasoning for such bin-packing problems.

Keywords: Constraint Programming, Global Constraints, Bin-Packing

1 Introduction

The `BinPacking` ($[X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m]$) global constraint captures the situation of allocating n indivisible weighted items to m capacitated bins:

- X_i is an integer variable representing the bin where item i , with strictly positive integer weight w_i , is placed. Every item must be placed *i.e.* $Dom(X_i) \subseteq [1..m]$.
- L_j is an integer variable representing the sum of items weights placed into that bin.

The constraint enforces the following relations:

$$\forall j \in [1..m] : \sum_{i | X_i = j} w_i = L_j$$

The initial filtering algorithm proposed for this constraint in [7] essentially filters the domains of the X_i using a knapsack-like reasoning to detect if forcing an item into a particular bin j would make it impossible to reach a load L_j for that bin. This procedure is very efficient but can return false positive saying that an item is OK for a particular bin while it is not. A failure detection algorithm was also introduced in [7] computing a lower bound on the number

of bins necessary to complete the partial solution. This last consistency check has been extended in [2]. Cambazard and O’Sullivan [1] propose to filter the domains using an LP arc-flow formulation.

In classical bin-packing problems, the capacity of the bins $\max(L_j)$ are constrained while the lower bounds $\min(L_j)$ are usually set to 0 in the model. This is why existing filtering algorithms use the upper bounds of the load variables $\max(L_j)$ (*i.e.* capacity of the bins) and do not focus much on the lower bounds of these variables $\min(L_j)$.

Recently [6] introduced an additional cardinality based filtering counting the number of items in each bin. We can view this extension as a generalization **BinPacking** $([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m], [C_1, \dots, C_m])$ of the constraint where C_j are counting variables, that is defined by $\forall j \in [1..m] : C_j = |\{i | X_i = j\}|$. This formulation for the **BinPacking** constraint is well suited when

- the lower bounds on load variables are also constrained initially $\min(L_j) > 0$,
- the items to be placed are approximately equivalent in weight (the bin-packing is dominated by an assignment problem), or
- there are cardinality constraints on the number of items in each bin.

The idea of [6] is to introduce a redundant global cardinality constraint [4]:

$$\begin{aligned} \text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m], [C_1, \dots, C_m]) \equiv \\ \text{BinPacking}([X_1, \dots, X_n], [w_1, \dots, w_n], [L_1, \dots, L_m]) \wedge \\ \text{GCC}([X_1, \dots, X_n], [C_1, \dots, C_m]) \end{aligned} \quad (1)$$

with a specialized algorithm used to adjust the upper and lower bounds of the C_j variables when the bounds of the L_j ’s and/or the domains of the X_i ’s change. Naturally the tighter are the bounds computed on the cardinality variables, the stronger will be the filtering induced by the GCC constraint.

We first introduce some definitions, then we recall the greedy algorithm introduced in [6] to update the cardinality variables.

Definition 1. We denote by pack_j the set of items already packed in bin j : $\text{pack}_j = \{i | \text{Dom}(X_i) = \{j\}\}$ and by cand_j the candidate items available to go in bin j : $\text{cand}_j = \{i | j \in \text{Dom}(X_i) \wedge |\text{Dom}(X_i)| > 1\}$. The sum of the weights of a set of items S is $\text{sum}(S) = \sum_{i \in S} w_i$.

As explained in [6], a lower bound on the number of items that can be additionally packed into bin j can be obtained by finding the size of the smallest cardinality set $A_j \subseteq \text{cand}_j$ such as $\text{sum}(A_j) \geq \min(L_j) - \text{sum}(\text{pack}_j)$. Then we have $\min(C_j) \geq |\text{pack}_j| + |A_j|$. Thus we can filter the lower bound of the cardinality C_j as follows:

$$\min(C_j) \leftarrow \max(\min(C_j), |\text{pack}_j| + |A_j|).$$

This set A_j is obtained in [6] by scanning greedily elements in cand_j with increasing weights until an accumulated weight of $\min(L_j) - \text{sum}(\text{pack}_j)$ is reached. It can be done in linear time assuming the items are sorted initially by weight.

Example 1. Five items with weights 3, 3, 4, 5, 7 can be placed into bin 1 having a possible load $L_1 \in [20..22]$. Two other items are already packed into that bin with weights 3 and 7 ($|pack_1| = 2$ and $l_1 = 10$). Clearly we have that $|A_1| = 2$ obtained with weights 5, 7. The minimum value of the domain of the cardinality variable C_1 is thus set to 4.

A similar reasoning can be used to filter the upper bound of the cardinality variable $\max(C_j)$.

This paper further improves the cardinality based filtering, introducing

1. In Section 2, an algorithm computing tighter lower/upper bounds on the cardinality variables C_j of each bin j , and
2. In Section 3, an algorithm to update the load variables L_j based on the cardinality information.

The new filtering is experimented on the Balanced Academic Curriculum Problem in Section 4.

2 Filtering the cardinality variables

The lower (upper) bound computation on the cardinality C_j introduced in [6] only considers the possible items $cand_j$ and the minimum (maximum) load value to reach *i.e.* $\min(L_j)$ ($\max(L_j)$). Stronger bounds can possibly be computed by also considering the cardinality variables of other bins. Indeed, an item which is used for reaching the minimum cardinality or minimum load for a bin j , may not be usable again for computing the minimum cardinality of another bin k as illustrated on next example:

Example 2. A bin j can accept items having weights 3, 3, 3 with a minimum load of 6 and thus a minimum cardinality of 2 items. A bin k with a minimum load of 5 can accept the same items plus two items of weight 1. Clearly, the bin k can not take more than one item with weight 3 for computing its minimum cardinality because it would prevent the bin j to reach its minimum cardinality of 2. Thus the minimum cardinality of bin k should be 3 and not 2 as would be computed with the lower bound of [6].

Minimum Cardinality of bin j Algorithm 1 computes a stronger lower bound also taking into account the cardinality variables of other bins $\min(C_k) \forall k \neq j$.

It prevents to reuse again an item if it is required for reaching a minimum cardinality in another bin. This is achieved by maintaining for every other bin k the number of items this bin is ready to give without preventing it to fulfill its own minimum cardinality requirement $\min(C_k)$.

Clearly if a bin k must pack at least $\min(C_k)$ items and has already packed $|pack_k|$ items, this bin can not give more than $|cand_k| - (\min(C_k) - |pack_k|)$ items to bin j . This information is maintained into the variables *availableForOtherBins_k* initialized at lines 5.

Example 3. Continuing on Example 2, bin j will have $availableForOtherBins_j = 3 - (2 - 0) = 1$ because this bin can give at most one of its item to another bin.

Since items are iterated in decreasing weight order at line 7, the other bins accept to give first their "heaviest" candidate items. This is an optimistic situation from the point of view of bin j , justifying why the algorithm computes a valid lower bound on the cardinality variable C_j . Each time an item is used by bin j , the other bins (where this item was candidate) reduce their quantities $availableForOtherBins_k$ since they "consume" their flexibility to give items. If at least one other bin k absolutely needs the current item i to fulfill its own minimum cardinality (detected at line 13), **available** is set to **false** meaning that this item can not be used in the computation of the cardinality of bin j to reach the minimum load.

On the other hand, if the current item can be used (**available=true**), then other bins which agreed to give this item have one item less available. The $availableForOtherBins_k$ numbers are decremented at line 22.

Finally notice that the algorithm may detect unfeasible situations when it is not able to reach the minimum load at line 28.

Algorithm 1: Computes a lower bound on the cardinality of bin j

Data: j a bin index
Result: $binMinCard$ a lower bound on the min cardinality for the bin j

```

1  $binLoad \leftarrow \text{sum}(pack_j)$ ;
2  $binMinCard \leftarrow |pack_j|$ ;
3  $othersBins \leftarrow \{1, \dots, m\} \setminus j$ ;
4 foreach  $k \in othersBins$  do
5    $availableForOtherBins_k \leftarrow |cand_k| - (\min(C_k) - |pack_k|)$ ;
6 end
7 foreach  $i \in cand_j$  in decreasing weight order do
8   if  $binLoad \geq \min(L_j)$  then
9     break;
10  end
11   $available \leftarrow \text{true}$ ;
12  for  $k \in othersBins$  do
13    if  $k \in Dom(X_i) \wedge availableForOtherBins_k = 0$  then
14       $available \leftarrow \text{false}$ ;
15    end
16  end
17  if  $available$  then
18     $binLoad \leftarrow binLoad + w_i$ ;
19     $binMinCard \leftarrow binMinCard + 1$ ;
20    for  $k \in othersBins$  do
21      if  $k \in Dom(X_i)$  then
22         $availableForOtherBins_k \leftarrow availableForOtherBins_k - 1$ ;
23      end
24    end
25  end
26 end
27 if  $binLoad < \min(L_j)$  then
28   The constraint is unfeasible;
29 end

```

Maximum Cardinality The algorithm to compute the maximum cardinality is similar. The changes to bring to Algorithm 1 are:

1. The variable $binMinCard$ should be named $binMaxCard$

2. The items are considered in increasing weight order at line 7, and
3. The stopping criteria at line 8 becomes $\text{binLoad} + w_i > \max(L_j)$.
4. There is no feasibility test at lines 27 - 29.

Complexity Assuming the items are sorted initially in decreasing weights, this algorithm runs in $O(n \cdot m)$ with n the number of items and m the number of bins. Hence adjusting the cardinality of every bins takes $O(n \cdot m^2)$. This algorithm has no guarantee to be idempotent. Indeed the bin j may consider an item i as available, but the later adjustment of the minimum cardinality of another bin k may cause this item to be unavailable if bin j is considered again.

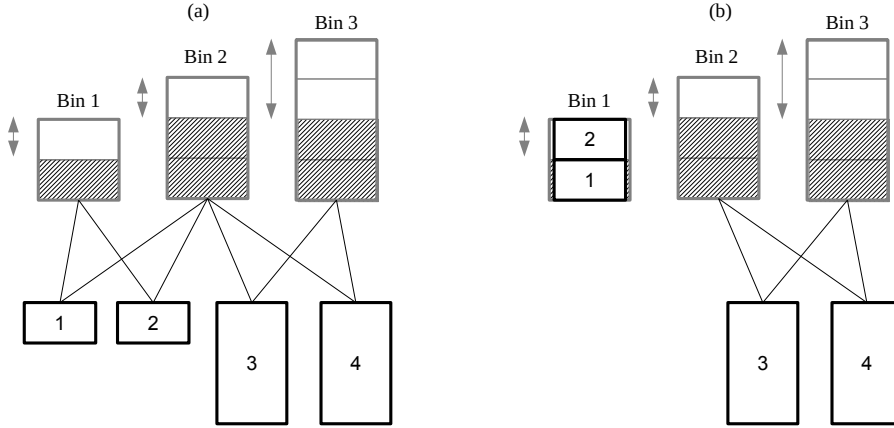


Fig. 1: (a) BinPacking instance with 3 bins and 4 items. The arcs represent for each item, the possible bins. (b) Domains resulting from the filtering induced with the tighter computation of the cardinalities.

Example 4. The instance considered - depicted in Figure 1 (a) - is the following:

$$\begin{aligned}
 &\text{BinPacking}([X_1, \dots, X_4], [w_1, \dots, w_4], [L_1, \dots, L_3]) \\
 &X_1 \in \{1, 2\}, X_2 \in \{1, 2\}, X_3 \in \{2, 3\}, X_4 \in \{2, 3\}, \\
 &w_1 = 1, w_2 = 1, w_3 = 3, w_4 = 3 \\
 &L_1 \in \{1, 2\}, L_2 \in \{2, 3\}, L_3 \in \{2, 4\}
 \end{aligned} \tag{2}$$

We consider first the computation of the cardinality of bin 2. This bin must have at least one item to reach its minimum load. We now consider the maximum cardinality of this bin. Items 1 and 2 can both be packed into bin 2 but doing so would prevent bin 1 to achieve its minimum load requirement of 1. Hence only one of these items can be used during the computation of the maximum cardinality for bin 2. Assuming that item 1 is used, the next item considered is item 3 having a weight of 3. But Adding this item together with item 1

would exceed the maximum load ($4 > 3$) (stopping criteria for the maximum cardinality computation). Hence the final maximum cardinality for bin 2 is one. The cardinality reasoning also deduces that bin 1 must have between one and two items and bin 3 must have exactly one item. Based on these cardinalities, the global cardinality constraint (GCC) is able to deduce that item 1 and 2 must be packed into bin 1. This filtering is illustrated on Figure 1 (b).

The algorithm from [6] deduces that bin 2 must have between one and two items (not exactly one as the new filtering). The upper bound of two items is obtained with the two lightest items 1 and 2. As for the new algorithm, it deduces that bin 1 must have between one and two items and bin 3 must have exactly one item. Unfortunately, the GCC is not able to remove any bin from the item's domains based on these cardinality bounds. Thus, this algorithm is less powerful than the new one.

Algorithm 2: Computes a lower bound on load of bin j

```

Data:  $j$  a bin index
Result:  $binMinLoad$  a lower bound on the load of bin  $j$ 
1  $binCard \leftarrow |pack_j|$ ;
2  $binMinLoad \leftarrow sum(pack_j)$ ;
3  $othersBins \leftarrow \{1, \dots, m\} \setminus j$ ;
4 foreach  $k \in otherBins$  do
5    $availableForOtherBins_k \leftarrow |cand_k| - (\min(C_k) - |pack_k|)$ ;
6 end
7 foreach  $i \in cand_j$  in increasing weight order do
8   if  $binCard \geq \min(C_j)$  then
9     break;
10  end
11   $available \leftarrow true$ ;
12  for  $k \in othersBins$  do
13    if  $k \in Dom(X_i) \wedge availableForOtherBins_k = 0$  then
14       $available \leftarrow false$ ;
15    end
16  end
17  if  $available$  then
18     $binMinLoad \leftarrow binMinLoad + w_i$ ;
19     $binCard \leftarrow binCard + 1$ ;
20    for  $k \in othersBins$  do
21      if  $k \in Dom(X_i)$  then
22         $availableForOtherBins_k \leftarrow availableForOtherBins_k - 1$ ;
23      end
24    end
25  end
26 end
27 if  $binCard < \min(C_j)$  then
28   The constraint is unfeasible;
29 end

```

3 Filtering the load variables

We introduce a filtering of the load variable taking the cardinality information into account. No such filtering was proposed in [6]. Algorithm 2 is similar to Algorithm 1 except that we try to reach the minimum cardinality requirements by choosing first the "lightest" items until the minimum cardinality $\min(C_j)$ is reached (line 8). Again a similar reasoning can be done to compute an upper bound on the maximum load.

limit(s)	A	B	C
15	13	27	41
30	18	34	46
60	21	37	51
120	25	43	57
1800	37	62	69

Table 1: Number of instances for which it was possible to prove optimality within the time limit.

4 Experiments

The Balanced Academic Curriculum Problem (BACP) is recurrent in Universities. The goal is to schedule the courses that a student must follow in order to respect the prerequisite constraints between courses and to balance as much as possible the workload of each period. Each period also has a minimum and maximum number of courses. The largest of the three instances available on CSPLIB (<http://www.csplib.org>) with 12 periods, 66 courses having a weight between 1 and 5 (credits) and 65 prerequisites relations, was modified in [5] to generate 100 new instances³ by giving each course a random weight between 1 and 5 and by randomly keeping 50 out of the 65 prerequisites. Each period must have between 5 and 7 courses. As shown in [3], a better balance property is obtained by minimizing the variance instead of the maximum load. For each instance, we test three different filtering configurations for bin-packing:

- A: The **BinPacking** constraint from [7] + a **GCC** constraint,
- B: A + the cardinality filtering from [6],
- C: A + the cardinality filtering introduced in this paper.

instance	time (ms)			best bound			number of failures		
	A	B	C	A	B	C	A	B	C
inst2.txt	timeout	timeout	679	3243	3247	3237	835459	1064862	829
inst14.txt	timeout	45625	6925	3107	3105	3105	1043251	228294	8530
inst22.txt	timeout	13971	281	3045	3041	3041	811852	48482	353
inst30.txt	timeout	118964	192	3416	3402	3402	795913	707487	129
inst36.txt	timeout	timeout	337	2685	2685	2671	847641	915849	364
inst47.txt	timeout	timeout	112	3309	3309	3303	2561038	3812512	269
inst65.txt	timeout	timeout	222	3416	3414	3402	921694	1091396	168
inst70.txt	timeout	timeout	101060	3043	3043	3041	1917729	1516627	125270
inst87.txt	16275	15089	251	3643	3643	3643	109173	65493	207
inst98.txt	timeout	timeout	48	2987	2987	2979	7023383	8261509	261

Table 2: Detailed statistics obtained on some significant instances.

Table 1 gives the number of solved instances for increasing timeout values. Table 2 illustrates the detailed numbers (time, best bound, number of failures) for

³ Available at <http://becool.info.ucl.ac.be/resources/bacp>

some instances with a 30 minutes timeout. As can be seen, the new filtering allows to solve more instances sometimes cutting the number of failures by several order of magnitudes.

5 Conclusion

We introduced stronger cardinality bounds on the **BinPacking** constraint by also integrating the cardinality requirements of other bins during the computation. These stronger bounds have a direct impact on the filtering of placement variables through the **GCC** constraint. The improved filtering was experimented on the BACP allowing to solve more instances and reducing drastically the number of failures on some instances.

References

1. Hadrien Cambazard and Barry OSullivan. Propagating the bin packing constraint using linear programming. In *Principles and Practice of Constraint Programming-CP 2010*, pages 129–136. Springer, 2010.
2. Julien Dupuis, Pierre Schaus, and Yves Deville. Consistency check for the bin packing constraint revisited. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 117–122. Springer, 2010.
3. Jean-Noël Monette, Pierre Schaus, Stéphane Zampelli, Yves Deville, and Pierre Dupont. A CP approach to the balanced academic curriculum problem. In *Seventh International Workshop on Symmetry and Constraint Satisfaction Problems*, volume 7, 2007.
4. Jean-Charles Régin. Generalized arc consistency for global cardinality constraint. In *Proceedings of the thirteenth national conference on Artificial intelligence-Volume 1*, pages 209–215. AAAI Press, 1996.
5. Pierre Schaus et al. *Solving balancing and bin-packing problems with constraint programming*. PhD thesis, PhD thesis, Universit catholique de Louvain Louvain-la-Neuve, 2009.
6. Pierre Schaus, Jean-Charles Régin, Rowan Van Schaeren, Wout Dullaert, and Birger Raa. Cardinality reasoning for bin-packing constraint: application to a tank allocation problem. In *Principles and Practice of Constraint Programming*, pages 815–822. Springer, 2012.
7. Paul Shaw. A constraint for bin packing. In *Principles and Practice of Constraint Programming-CP 2004*, pages 648–662. Springer, 2004.